Prelude

A typical MSc thesis will be structured according to a fairly standard sequence of sections, and one example is shown here. However, it is hard and perhaps even counter-productive to generalise: the goal of this document is *not* to be prescriptive, to show you how you *must* structure your thesis; rather it is to simply to act as a guideline, to show you how you *could* structure your thesis. In particular, each chapter's indicated page-count is important but *not* absolute: the aim is simply to highlight that a clear and concise description is better than a rambling alternative that makes it hard to separate important content and facts from irrelevant trivia.

You can use this document as a LaTeX-based template for your own thesis by simply deleting extraneous sections and content, and adding your own text. If you do that, we recommend using BibTeX to deal with the associated bibliography. You may want to use Overleaf (an online cloud-based collaborative LaTeXplatform).

The words "thesis" and "dissertation" are used interchangeably here. All active hyperlinks in the PDF file are rendered in the same shade of dark blue.

Dissertation Type: research

DEPARTMENT OF ENGINEERING MATHEMATICS

COVID-19 Fake News and Health Misinformation Detection on Twitter

Ali Almalki

Supervised by: Dr. Ryan McConville

A dissertation submitted to the University of Bristol in accordance with the requirements of the degree of Master of Science in the Faculty of Engineering.

Monday 4th October, 2021

Declaration

This dissertation is submitted to the University of Bristol in accordance with the requirements of the degree of MSc in the Faculty of Engineering. It has not been submitted for any other degree or diploma of any examining body. Except where specifically acknowledged, it is all the work of the Author.

Ali Almalki, Monday 4th October, 2021

Contents

1 Contextual Background **1**
1.1 Introduction . . . 1
1.2 Problem Statement . . . 1
1.3 Motivation . . . 2
1.4 Methodology . . . 3
1.5 Objectives . . . 3
1.6 Challenges . . . 4
1.7 Summary . . . 4

2 Technical Background **5**
2.1 Machine Learning and NLP Background . . . 5
2.2 Word Embeddings Algorithms . . . 7
2.3 Graph Terminology and Representation . . . 8
2.4 Representation Learning on Graphs . . . 9
2.5 Supervised Machine Learning Classifications Algorithms . . . 10
2.6 Fake News Detection Relevant Literature . . . 13
2.7 Dataset . . . 15

3 Project Implementation **17**
3.1 Introduction . . . 17
3.2 Project Management . . . 17
3.3 Dataset Creation . . . 18
3.4 Dataset Overview . . . 19
3.5 Architecture of the Experimental Setup . . . 22
3.6 Text Modelling . . . 25
3.7 Graph Modelling . . . 26
3.8 Evaluation Methods . . . 28

4 Project Results and Critical Evaluation **31**
4.1 Introduction . . . 31
4.2 Approach to Result Generation . . . 31
4.3 Baseline Model Results . . . 31
4.4 Removing Hashtag Keywords Results . . . 32
4.5 ML Models with Independent Features Results . . . 32
4.6 ML Models with Combined Features Results . . . 34
4.7 ML Models with Node2Vec Feature Representation Results . . . 36
4.8 Results Analysis . . . 36
4.9 Performance summary and conclusion . . . 37
4.10 Results Evaluation . . . 37
4.11 Assessment with regards to project objectives . . . 39

5 Conclusion **41**
5.1 Summary . . . 42
5.2 Project Status . . . 42
5.3 Further Work . . . 43

List of Figures

1.1 A tweet portraying false information that can mislead and manipulate many individuals who believe this statement is true [20]. 3

2.1 A figure illustrating node embedding learning methodology where we have the encoder (ENC) that maps nodes to a low-dimensional embedding space in a way that show the relative positions of the nodes in the original graph. [17] 9

3.1 A screenshot showing the repository status on my local machine 17
3.2 A screenshot showing the repository log on my local machine 18
3.3 The first 10 rows of the dataset 19
3.4 Nested JSON Dataset Structure 19
3.5 A bar chart showing label count 20
3.6 A sample of the tweet text 21
3.7 Tweet text length for each label 21
3.8 Most frequent words in the dataset 22
3.9 The architecture of the proposed pipeline. 23
3.10 Important features selection. 25
3.11 A graph representing the tweet ID and its retweets/replies connections 26

4.1 Usage of Word2Vec features generates the most accurate results 33
4.2 Comparison of graph-based and text-based modelling 38

List of Tables

3.1 The classifiers with various independent feature representation 26
3.2 Confusion matrix. 28

4.1 DT Classifier - Baseline Model 31
4.2 After removing hashtag keywords - DT Classifier 32
4.3 BoW-based feature with various classifiers 32
4.4 TF-IDF based feature with various classifiers 33
4.5 Word2Vec-based feature with various classifiers 33
4.6 Combined features with SVM classifier 34
4.7 Combined features with MNB classifier 34
4.8 Combined features with GNB classifier 35
4.9 Combined features with DT classifier 35
4.10 Combined features with KNN classifier 36
4.11 Node2Vec-based feature with various classifiers 36
4.12 Average performance for all classifiers using independent features 38
4.13 Average performance for all classifiers using a combination of features 38

Notation and Acronyms

NLP	:	Natural Language Processing
ML	:	Machine Learning
JSON	:	JavaScript Object Notation
CSV	:	Comma-Separated Values
BoW	:	Bag of Words
TF-IDF	:	Term Frequency — Inverse Document Frequency
Word2Vec	:	Words to Vectors
Node2Vec	:	Nodes to Vectors
NB	:	Naïve Bayes
MNB	:	Multinomial Naïve Bayes
GNB	:	Gaussian Naïve Bayes
DT	:	Decision Tree
KNN	:	K-Nearest Neighbours
SVM	:	Support Vector Machines
LSVM	:	Lagrangian Support Vector Machines
ROC	:	Receiver Operating Characteristic Curve
GNN	:	Graph Neural Network
G	:	The Graph
V	:	Vertices (Nodes)
E	:	Edges
(A, W)	:	(Adjacency, Weight)
u	:	Feature Vector

Executive Summary

During the COVID-19 pandemic, social networks have seen a substantial amount of false news [28]. Furthermore, people have been discussing numerous false remedies to cure COVID-19. However, these remedies are extremely dangerous to human health. The director-general of the World Health Organization calls it “infodemic" [31] because of the amount of misinformation, disinformation, and "false news" relating to COVID-19. With the enormous number of news regarding COVID-19 on the Internet, it is difficult for many to assess truthfulness. Moreover, the riots and panic shopping also occurred due to the propagation of “false news" [31]. In this thesis project, I aim to build an automated COVID-19 misinformation detection system and investigate the value of a social network structure compared to the text-based classification approach. I have implemented a variety of techniques to detect fake news and misinformation in tweets related to COVID-19. The research objective is to classify each tweet as either true/fake with various text feature representations techniques and graph structure to compare and evaluate their performances. The project is comprised of two parts which are text-based and graph structure-based fake news detection techniques. For the first part, I conduct five different classification algorithms relying on various embeddings techniques including BoW, TF-IDF and Word2Vec embeddings. For the second part, I represent the data in a graph structure and learn the feature representations for the nodes using the Node2Vec embedding algorithm, which can then be used for the downstream classification task. Different studies [29, 32] revealed that N-grams based features are efficient in identifying false information. The author in [33] uses TF-IDF feature to classify hoax news with 84.67% accuracy. The classifiers used in this paper are more modern KNN, MNB, LSVM, and logistic classification. In this thesis, it was concluded that the use of a combination of features in the GNB classifier has the most accurate performance. Moreover, text-based modelling outperforms graph-based modelling in terms of ROC score, accuracy as well as weighted average F1 score.
The overall aim was to produce a system that can auto-detect fake news in tweets related to COVID-19, improving the results obtained by previous studies.

The main achievements of the project are as follows:

- I learnt how to clean and prepare large scale data in JSON format and convert it into a suitable format to be ready for the study analysis.
- I wrote a total of 18898 lines of source code (in Python), implementing all the experiments analysed in this thesis.
- I learnt how to build an NLP pipeline including word tokenisation and removing stopwords from the tweet text.
- I wrote functions in python code that encode different text feature representation including BoW, TF-IDF and Word2Vec.
- I investigated and trained many classification models with different feature representation combinations.
- I represented the data as a network structure (graph) and modelled it to be used in the context of detecting false information.
- I applied continuous feature representations for the nodes in a graph to be used for the classification task.

Supporting Technologies

Supporting technologies used in the project are as follows:

- I used *NumPy* and *Pandas* public-domain Python libraries for data manipulation and analysis.
- I used *JSON* built-in package in Python to work with JSON data format.
- I used *nltk* built-in package in Python for natural language processing tasks.
- I used *sklearn* built-in package in Python for machine learning and classification modelling.
- I used *sklearn.feature extraction.text* from sklearn package for applying different features techniques.
- I used *NetworkX* python library to create the social network graph.
- I used Google Drive for remote storage of data.
- I used Git version control system for project management and keeping track of my progress on my local machine. Specifically, I used these Git commands:
  - git init: to initialise a repository.
  - git status: to give information on the current status of the repository and its contents.
  - git add: to add files to the staging area.
  - git commit: to commit all staged changes.

Acknowledgements

Ali Almalki writes here to say huge thanks to his supervisor Dr. Ryan McConville for his great domain knowledge and consistent support. The skills and knowledge which I have gained throughout the thesis project I perceive as a very valuable component in my future career development.

Chapter 1

Contextual Background

1.1 Introduction

The pandemic has been accompanied by a massive wave of false news that made it more difficult for the public to comprehend the truth. People use different social networks to share information and communicate with each other. However, this also opens a room to propagate fake information on these platforms. The spread of fake news can be extremely dangerous, especially when facing the COVID-19 epidemic. Moreover, fake news about fake remedies to cure COVID-19 can be hazardous to human health and lead to human death [43]. Processing an enormous amount of text manually on these social media platforms, particularly Twitter, is troublesome. Therefore, it is essential to present automatic methods to analyze text and annotate if the news is fake or real. Fake or inaccurate information can also put a human body at risk of exposure to viruses and also cause anxiety, mental stress [40]. Besides, the purpose of spreading fake news can also be to mislead people to gain attention. Believing wrong information can be an easy task; however, refuting fake news is a challenging task – not many people ask for evidence to back up the statement.

In the digital era, the general public can be easily misled by fake news. This can be due to various reasons, such as lack of education and lack of interest in news verification. However, the propagation of fake news can be dangerous. Misinformation on COVID-19 can encourage individuals not to wear masks, take certain precautions, or even get a vaccine when these are the factors that can keep them safe. Public health messages should be considered since fake information can be posted as very concise headlines on the internet, especially on Twitter. Therefore, it is crucial to automatically differentiate real and fake news related to COVID-19 to avoid hazardous consequences to public health.

A recent study [39] reported that 1500 tweets were posted on Twitter regarding COVID-19, including 1274 were completely false and 226 were partially false. This implies that it may be unknown to the individuals that they are spreading fake news. If they knew that the shared information was fake, then it is probable that they would have preferred sharing true news. Hence, It is beneficial to understand the essence of propagating misinformation phenomenon on these social media platforms, especially Twitter. Twitter allows its users to share ideas easily, and posting a tweet or re-posting a particular tweet on Twitter does not require much effort. Nevertheless, words can be misused, and stories can be made up to make everyone believe using conspiracy theories. Research [34] shows that the number of conspiracy theories has increased due to the spread of COVID-19 worldwide. Another study [2] linked 5G technology with COVID-19, this has become dangerous since several people have started to burn the 5G towers in the United Kingdom. Finally, another study revealed that fake news on Twitter gets more response and spread faster than real news [2].

1.2 Problem Statement

The COVID-19 pandemic has caused many restrictions and induced different issues that impact people's social and economic lives. Employees have stopped in office work due to the fear of COVID-19. The root of this issue has become even more profound due to social media amplifying fake news. Moreover, the COVID-19 pandemic has become one of the significant events in all of our lives, and the impact has also been enormous due to approximately 199,376,990 million people being infected from the COVID-19 [1].

Rumours, false news, false data, lies, and gossips related to COVID-19 can provoke the public's actions because it is harming immune systems and physical health. This encourages people to believe in the conspiracies they see or hear on Twitter. Social media has become a medium that could negatively affect the public's mental health as well. For example, a recent study [4] demonstrated that 800 people have lost their lives globally, and 5,800 have been admitted to hospitals due to the fake news about COVID-19. Many individuals believed that the cure for the COVID-19 is by drinking methanol or alcohol-based cleaning products, not knowing how harmful these substances are to the body.

To provide reliable information, different research-led websites can make a big impact. For instance, more than 81 million articles are linked to the website of the Johns Hopkins Coronavirus Resource Center on Google. A research of 1000 participants about fake reports carried out in 2020 reveals that individuals prefer to readily spread news without being certain that it is genuine or false [25]. If the similar partakers realize which news is false. However, they are more likely than fake to share (genuine) news. This shows that false news is very desirable so that credible information may be distributed only.

COVID-19 has been a serious concern, and this gives some people an opportunity to introduce different kinds of fake news on social network platforms like Twitter. Some of the theories presented that the Chinese deliberately made COVID-19 to start a war against the USA. However, as time passed, many people beginning believing the whole idea of COVID-19 is not true, and it is merely a hoax by the governments – this idea has been exaggerated to create a sense of fear in people's minds [10]. Various techniques [3] have been proposed to identify fake news related to COVID-19 on Twitter automatically. This is a necessary step to stop or mitigate the storm of fake news on social media platforms. Nonetheless, the research on this topic is still evolving. Hence, it is beneficial to propose and study a new approach where I use both text-based and graph-structure based techniques that can detect false news on tweets regarding COVID-19. Both the content of the tweet (text) and the spreading of the tweet (the graph) would be useful indicators to detects fake tweets. The text contains keywords or phrases that indicate false information whereas the graph network structure could reveal how fast the tweet spreads as well as the most influential tweets. This can be identified by how many retweets/replies a tweet receives.

1.3 Motivation

The rationale behind this thesis study was derived from the considerable damage induced by the circulation of fake news concerning COVID-19 on social media platforms, primarily Twitter, and its impact on society's awareness. The COVID-19 pandemic has caused havoc to several individuals' lives, negatively impacting health and mental state due to the fake news about COVID-19. The propagation of fake news about the epidemic on Twitter, which has hundreds of users, can be devastating as many people are unsure what to believe. Consequently, this study is designed to detect fake news on Twitter related to COVID-19.

This research aims to build a robust automated misinformation detection system that identifies fake tweets related to COVID-19. This is highly crucial because the general public should verify the truthfulness of a tweet's claims before sharing or commenting on the information. The invention of COVID-19 vaccines has started to kick in with scientists' help. The fear of fake news can undermine people's confidence in vaccines, so the urge to identify fake news automatically can help massively to combat this virus. With the help of this project, tweets regarding COVID-19 can be verified and examined to check their credibility. This research is essential for my understanding of the fake news relevant to COVID-19. This further emphasises the aim of this whole thesis project, which is to develop an independent system that assists in recognising fake news on Twitter. The disadvantages of spreading fake news have a tremendous emotional and physical impact on elderly people who tend to believe news and theories promptly without any cross-verification. This can become life-threatening for a particular age group unwilling to check the veracity of the news. Figure 1.1 shows a tweet portraying fake information that can mislead many people and can be manipulated by many individuals. Thus, detecting fake news with automatic methods on Twitter related to COVID-19 is of vital importance.

Figure 1.1: A tweet portraying false information that can mislead and manipulate many individuals who believe this statement is true [20].

1.4 Methodology

1.4.1 Research Question

This project studies the value of both text and a social network structure for an automated misinformation detection system. The main research question to explore in this thesis – is misinformation better detected using text, or can the social network graph structure itself be a better detector?

1.4.2 Research Hypothesis

In this research, the central hypothesis to test is whether an input of a node feature representation in a graph to a supervised classifier algorithm would yield better performance than text-based features in detecting fake tweets. To the best of my knowledge, this approach fills a gap in the literature.

1.4.3 Research Methodology

In this project, both traditional NLP techniques and social network analysis are applied and compared. The project consists of two main parts. The first part is the text-based classification technique, which involves building supervised classification algorithms. These classification models classify tweets as either true/fake based on tweet text content. This research will use various text features, such as BoW, TF-IDF, and Word2Vec, and compare their performances with different classification models.

The second part uses the graph-structure classification technique, where the data is represented in a graph containing nodes and edges (connections between the nodes). Then, this graph representation will be fed as input to supervised ML algorithms. This graph representation learning approach shows the relationships between the tweet contents and the key players in promoting the information, as well as identifying the reasons that promote the information. Therefore, data will be represented in a graph in which nodes represent tweets id and edges represent the relationships between them (retweets/replies). This will allow classifying the tweet as either true/fake based on graph structure and not the tweet text.

Moreover, the node classification technique will be used to classify the tweet in the output layer. Still, it needs further processing to get it working correctly.

1.5 Objectives

A list of objectives was provided to design an automated COVID-19 misinformation detection system that identifies tweets that include fake content. These objectives aim to create such a system that assists users in verifying the truthfulness of any COVID-19 related tweets. The main objectives of this research project are as follows:

the dataset is not labelled, adding binary labels to the dataset as true/fake is required. Building a baseline model that classify each tweet and improving its performance with coming experiments.

Encoding BoW, TF-IDF and Word2Vec feature representation from the tweet text.

- Training classifiers with independent feature representation.
- Training more than one feature to represent the text.
- Represent the tweet text related information in a graph structure
- Get input vectors from the graph
- Training classifiers with graph feature representation.
- Compare and analyse the performance of both text and graph approaches.
- Applying graph feature representation to a GNN algorithm.

1.6 Challenges

The main challenges of this project are listed as follows:

- Preparing and processing large scale dataset and processing data in the right form for each task in the study.
- labelling the tweet text.
- Representing the tweets in a graph (social network analysis).
- Training the classifiers takes a long time due to the large scale dataset.
- How to compare the performance of NLP methods which have access to more information on the labels.
- How to decide the graph structure contents since there is no best way to do it.

1.7 Summary

The objective of this project is to compare various feature representations including text-based and graph structure techniques along with several ML classification algorithms. The key contributions are as follows:

- Manage large scale and complex data in JSON format and converting it into a suitable format for the research experiments.
- Implement many feature representations to build an automated fake news classification system. These include BoW, TF-IDF, Word2Vec and Node2Vec embeddings algorithms.
- Apply NLP techniques to detect fake tweets.
- Representing a large scale data into a graph structure.
- Utilize graph structure method to detect fake tweets.
- Review the performance of the classification models with various feature representation combinations.
- Design experiments to analyse the effectiveness of each feature and algorithm.
- Analyse the results and conclude whether text-based or graph structure technique, is better.

[1]
[2]
[3]

[1] Oral Presentation YouTube Link: https://youtu.be/DFNR-0xEzbg
[2] GitHub Repository Link: https://github.com/AlmalkiHub/DSP
[3] Data Files Link on Google Drive: https://drive.google.com/drive/folders/13SLsFCMiw7zdNJx8z01iyX7-9Clfq9ZG?usp=sharing

Chapter 2

Technical Background

This chapter demonstrates the technical background and relevant literature on which the implementation of the project depends in Chapter 3. The rest of the chapter is organised as follows: First, the machine learning and natural language processing background which demonstrates basic concepts related to this field. Following this is an explanation for the different word embeddings algorithms that were used in the project implementation 3. Next, concepts and knowledge about data graph representation and some related research studies regarding COVID-19 fake news detection are presented. Finally, an overview of the data sets referenced in chapter 3 is introduced.

2.1 Machine Learning and NLP Background

2.1.1 Supervised Learning

Supervised Learning is a type of machine learning which is applied on a dataset which is labelled. This implies that in supervised learning, we learn from labelled data to predict the output. The value of this output variable in our case is known for the labelled data which is either true or fake tweet.

2.1.2 Unsupervised Learning

Unsupervised Learning is a machine learning task that identifies hidden patterns in the feature space based on unlabelled data. This means that the value of the output variable is not known while learning from the data. The model is responsible for drawing insights from the data itself such as the tweet text feature.

2.1.3 Bayes Theorem

Bayes theorem, alternatively known as the Bayes law, is one of the most prominent theorems in the field of probability and statistics. It helps in computing the conditional probability for an occurrence of an event. Conditional probability is defined as the updated probability of the occurrence of an event A given that some other event B has occurred and is denoted as P(A|B). It is computed as follows:

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \tag{2.1}$$

Likewise,

$$P(B|A) = \frac{P(A \cap B)}{P(A)} \tag{2.2}$$

It follows from the above formulae:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)} \tag{2.3}$$

Therefore, this theorem provides us with a way of computing the conditional probability for two event A and B without knowing their joint probability.

2.1.4 Gaussian Distribution

Gaussian Distribution is a kind of distribution with a bell-shaped curve. This distribution is symmetric about both sides mean showing that the data centered around the mean is more frequent than the data lying away from the mean. The two parameters of this distribution are mean and standard deviation. It is known to have a mean of 0 and standard deviation of 1 and for any distribution, it can be converted to a Gaussian Distribution by using the given formula:

$$z = \frac{x - \mu}{\sigma} \tag{2.4}$$

Where μ is mean and σ is the standard deviation.

When considered as a probability distribution or a probability mass function, the Gaussian Distribution is computed as follows:

$$\mathrm{f(x)} = \frac{e^{\frac{-(x-\mu)^2}{2\sigma^2}}}{\sigma\sqrt{2\pi}}$$

Dot product:

The dot product of two vectors is denoted by a ".". It is a vector operation that takes as input two inputs and returns a single numeric value which is not a vector. It is computed as follows:

$$p.q = |p||q| \cos(p, q) \tag{2.5}$$

where |p| and |q| represent the magnitudes of these vectors. The magnitude of a vector which computed as shown below:

$$|p| = \sqrt{\sum_{i=1}^{N} (p_{i2} - p_{i1})^2} \tag{2.6}$$

where N is the number of dimensions of a vector and *i2* and *i1* are the values of terminal and initial points of that vector respectively in terms of a component or dimension *i*.

2.1.5 Tokenisation

Token is considered as the smallest meaningful unit one considers in the context of a given natural language related problem. It can be a single word, n-grams or sentences i.e. A document can be tokenized into sentences and messages can be tokenized into words. The most common tokenization technique is the conversion of the text corpus into individual words. At time, the size of a token may be as small as a single character.

2.1.6 Normalization of Text

Tweet text consists of unstructured data, and it must be processed such that it helps the machine to learn from it. To accomplish this, it must be converted into some standard format [21, 35]. All the words consist of a root word and an additional inflectional part i.e., for the word "Vaccination", the root word is "Vaccin", and the inflectional part is "ation". Normalization is the process of reducing a word into its base or root form by reducing the inflectional part to limit the size of vocabulary, the number of unique words in text corpus. Two of the most used methods [5] for this purpose are defined as follows:

- **Stemming**: Stemming is the method which just removes the last few characters in a word to reduce it to its base or root form thereby mapping a group of words to the same stem word even though the stem might not be a valid word of the language. Such kind of normalization can be erroneous at times i.e it reduces both "Cars" and "Caring" to car even though they have totally different meanings.
- **Lemmatization:** Lemmatization reduces the inflected words to a single word thereby making sure that the root word is a part of the actual language. This root word is often known as Lemma. Therefore, it is often used where it is necessary to get the valid words of a language i.e it reduces the words "Running", "Ran" and "Runs" to the same root word "Run" which is a valid word of English language.

2.2 Word Embeddings Algorithms

This section illustrates the word embeddings techniques applied in the project implementation in Chapter 3 to represent the tweet text into numeric vectors.

2.2.1 Bag of Words

BoW is a very popular technique used in Natural Language Processing for extracting features from text. Its popularity in text modelling has resulted from its simplicity and the fact that it is computationally inexpensive. It represents a document or a text source as a function of the frequency of tokens occurring in it. Its name is inspired by the fact that it does not consider the order of tokens in the document. Rather than that, it gives importance to their occurrence [46]. The Bag of Words algorithm comprises of the three following steps:

- Initially, the vocabulary is determined from the text corpus. Vocabulary is the number of unique words in the corpus.
- In the second step, a document vector is constructed which has length equal to the length of the vocabulary and each index in that vector maps to a particular word in the vocabulary. The index corresponding to a token is assigned a value which is equal to the frequency of that token for that document.

The Bag of Words technique is generally used after applying normalization on text.

2.2.2 Term Frequency-Inverse Document Frequency

TF-IDF is a popular technique used in Text Mining and Information Retrieval to determine the relevance of a word in a document. Term Frequency refers to the frequency of occurrence of a word in a document or measure of the weightage of the word in a document. The sole use of the raw count of the number of occurrences of a word cannot be reliable in determining its relevance as there exist certain words which occur commonly across documents but do not aid in topic modelling of the document. It is due to this very reason that an added metric called Inverse Document Frequency is used to refine the measure of relevance [35]. It manifests the commonality or rarity of a word across a set of documents and the more common a word is, the smaller the value of Inverse Document Frequency is. The TF-IDF is computed as follows:

$$TF - IDF = TF.IDF \tag{2.7}$$

Here,

$$\text{TF} = \frac{\text{Number of occurrences of the word in the document}}{\text{Number of words in a document}} \tag{2.8}$$

$$\text{IDF} = \log\left(\frac{\text{Number of Documents}}{\text{Number of documents containing the word}}\right) \tag{2.9}$$

While computing the term frequency, we compute the weight of a term in a document by dividing its raw count in a document by the total number of terms in the document as a convention. While computing the inverse document frequency, logarithm is applied on the result of the fraction to dampen or smooth the effect of IDF on the TF-IDF value so that high values of IDF do not have a large impact on the TF-IDF value.

2.2.3 Word2Vec

Word2Vec is a yet another popular method of representing the document vocabulary in the form of feature vectors. It is computationally expensive since it requires training a shallow neural network architecture, but the performance gain provided by this method is very significant since it can identify which words are semantically similar and capture the context of every token in the document vocabulary. To train a neural network to learn word features, the features are initially represented in one-hot encoding. In this format, each word is represented by a vector of length equal to the size of the vocabulary. The vector has zero at all indices and one at the index corresponding to the word it is supposed to represent. In this technique, the shallow network is trained for a language-based problem which is not necessarily relevant to the problem we are attempting to solve such as auto completion of text. Once the model is trained for such a problem, it automatically learns the semantic relationship between the words. Consequently, the weights of this model can be used as the representations of tokens. The Word2Vec method has two variations [27]. The variations and their details are mentioned as follows:

- **Common Bag of Words:** In the CBoW method, the context of a given target word is used to predict that word. Context of a word refers to the surrounding n-grams of a word. The input layer has a dimension equal to kxV where k is the number of context words takes and V is the number of unique words in the vocabulary. The output layer has a dimension equal to V. The one-hot representations of each word are fed into a separate input layer. These representations are then multiplied with corresponding weight matrices and fed into the hidden layer. The weight matrix corresponding to the output of the hidden layer serves as the word representation which is extracted after training the shallow neural network.

- **Skip-Gram model:** The input and output dimensions of the skip-gram model is opposite to the CBoW model. In this case, we use a specific word to predict the context. The input layer has a dimension equal to V, where V is a specific word. The output layer has a dimension of kxV where k is the number of tokens in the context and V is the size of vocabulary. In case of this model, the weight matrix between the input layer and the hidden layer is used as the representation for each word after training the model.

2.3 Graph Terminology and Representation

This section illustrates some basic knowledge about data graph representation.

2.3.1 Graph Representation

Graph representation comprises data that are connected to each other. The critical point to highlight is that graph data is similar to any other data type. However, the main difference between graph data and other data types is that graph data is structured data. This means that in addition to defining the data points, the relation between the data points should be defined as well, which provides us with a structure. This extra information is often valuable in analyzing the data, so graphs and graph neural networks represent better performance than other data structures.

2.3.2 Nodes and Edges

Nodes are the data points that are represented in the graph. What makes graph data different from other types of data is the representation of edges that connect to nodes or data points that are similar. Hence, if two nodes are related or identical, there will be an edge connecting the nodes. In this context, the nodes could present the tweet text, and the edges are the communication between Twitter users by retweeting/replying to a given tweet.

2.3.3 Features

The recent papers in the neural network domain [37, 24] allow us to introduce a piece of additional information in the form of features. These features are the extra information that each node could carry on. This information in the form of a feature could be the number of followers, likes, total replies, etc.

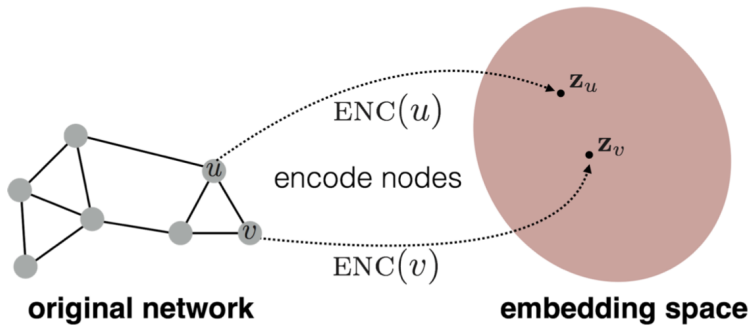

Figure 2.1: A figure illustrating node embedding learning methodology where we have the encoder (ENC) that maps nodes to a low-dimensional embedding space in a way that show the relative positions of the nodes in the original graph. [17] .

2.3.4 Directed Graph

In the case of a directed graph, an arrow shows how the nodes are related in this case. For example, node A is connected to node B, but node B is not connected to node A. So what we do is we draw an arrow from the source (Node A) to the target (Node B). Twitter is an example of a directed graph in the social media network where if the first person follows the second person, the second person does not need to follow back or follow the first person. To mathematically represent graph data, this can be shown in the following notation:

$$G = (V, E, u) \tag{2.10}$$

where G is the graph, V describes vertices (nodes), and E is a vector that describes edges or links, including two types of the matrices adjacency matrix and weights matrix. u represents the feature vector.

2.3.5 Undirected Graph

In an undirected graph, there are two arrows or no arrows at all to define an undirected graph. Moreover, in an undirected graph, the relation between the nodes is symmetric. This implies that when node A is connected to Node C, Node C is also connected to node A.

2.4 Representation Learning on Graphs

This part illustrates the approach behind learning node embeddings. Likewise, demonstrates the Node2Vec embedding algorithm which is used in the project implementation described in chapter 3

2.4.1 Learning Node Embeddings

One of the primary challenges faced in graph representation learning is to select an embedding method for the graph nodes [17]. The aim of learning the appropriate feature representation of nodes is to transform these nodes to an embedding space and use the resultant transformation of each node as a low-dimensional vector as an input to the classification algorithm. One of the most used approaches for learning node embeddings is the use of an encoder-decoder model. Figure 2.1 shows an overview of node embedding learning approach. The encoder part is responsible for mapping the nodes into the embedding space using a well devised methodology. Usually, a shallow embedding approach is employed where embedding function is simply a lookup function which extracts the embedding vector from the embedding matrix when given a unique node id using the strategy that had been devised to generate the embedding matrix. On the other hand, the decoder part of the network is used to map the feature representations or embeddings to the respective node or structural and positional information about that node such that the node and its neighborhood can be reconstructed using this information.

2.4.2 Node2Vec

Node2Vec is a representational learning method that is used to represent the nodes in a graph as vectors. It can be used to embed every node in the graph using second order random walks by setting every node as the target node one by one. The graphs are weighted. The random walk mechanism generates a sequence of nodes that are traversed probabilistically, and it terminates when the pre-defined walk-length (number of nodes in a traversal) is reached [13]. In a first order random walk, a node from the neighboring nodes must be picked based on the transition probability corresponding to each neighboring node which is computed using the edge weights leading to that node. Let n be the current node, then the probability of traversing a neighboring node m for a first order random walk is computed as follows:

$$P(m|n) = \frac{E(n,m)}{\sum_{i=1}^{N} E(n,m_i)} \tag{2.11}$$

Where N is the set of neighboring nodes for the current node N and E is the edge weight from n to m. However, the second-order random walk mechanism that is used in the Node2vec method is slightly different. It uses a function called alpha ($\propto$) which is multiplied by the edge weights in the equation for computing the transition probabilities in the first order random walk. These transition probabilities are known as second order transition probabilities and are computed as follows:

$$P(m|n) = \frac{\propto (p,q,t,m)E(n,m)}{\sum_{i=1}^{N} \propto (p,q,t,m)E(n,m_i)} \tag{2.12}$$

Here, the value of t determines whether the neighboring node was the last node traversed, it has been a part of the traversal previously or it has not been explored yet [13]. Thus, it can take three unique values. For the computation of $\propto$, two hyperparameters are taken, p and q. The value of p is used in the computation of $\propto$ when the neighboring node being considered as already been explored but it was not the previous node, and the value of q is used when it was the previous state, or the previous node traversed. The computation of $\propto$ can be summarized as follows:

$$\propto (p,q,t,m) = \frac{1}{p} if t_m = 1 \tag{2.13}$$

$$\propto (p,q,t,m) = \frac{1}{q} if t_m = -1 \tag{2.14}$$

$$\propto (p,q,t,m) = 1 if t_m = 0 \tag{2.15}$$

The set of nodes obtained by setting a node as a target node after performing the second order random walk are then passed through the Word2Vec algorithm to generate the embeddings or numerical representations of that target node.

2.5 Supervised Machine Learning Classifications Algorithms

2.5.1 Naive Bayes Algorithm

Naive Bayes is a probabilistic algorithm in the domain of Machine Learning that harnesses the power of the Bayes theorem. It is primarily termed as naive as it is primarily based on two assumptions [36] stated as follows:

- Each pair feature in the dataset are independent which is rarely the case in most real-world scenarios.
- Each feature is equally important in determining the outcome.

The assumption of independence implies that change in the value of one feature should not impact the value of the other features in the data. Both the assumptions are untrue in most of the real-world scenarios. In the context of a predictive modelling problem, the Bayes theorem can be summarized using the given equation:

$$P(y|X) = \frac{P(X|y)P(y)}{P(X)} \tag{2.16}$$

Here,

$$X = (x_1, x_2, x_3, \ldots, x_n) \tag{2.17}$$

The left-hand side of the equation is known as the posterior probability and on the right-hand side, P(y) is known as the prior probability and $P(X|y)$ is known as the likelihood. The term in the denominator is known as the probability of likelihood of evidence. For a classification problem, y can be termed as the class label and x1, x2, . . . , xn are termed as the input features. For a given instance of data, the Naive Bayes classifier uses the formula stated above to compute the likelihood of that instance belonging to a specific class label or a category [12]. The class outputted by the classifier as a prediction is the one with a greater likelihood as computed by the formula. It can have two variations:

- **Multinomial Naive Bayes:** The algorithm described above is essentially the multinominal naive bayes algorithm. An important feature for the multinomial naive bayes is that can be used to classify data belonging to more than two classes.

- **Gaussian Naive Bayes:** It is a variant of the Naive Bayes algorithm that is particularly used when the input features have continuous values. This algorithm assumes that values of these continuous features follow a Gaussian distribution. In this variant of the Naive Bayes algorithm, data is segmented according to class labels and the mean and standard deviation of every continuous feature is computed separately for each segment corresponding to a class label. Given a new instance, the likelihood in the Bayes theorem is computed using the probability density function of Gaussian distribution for each class using their mean and standard deviation. The class with the greater conditional probability is the one which the new instance belongs to.

2.5.2 Support Vector Machines

SVM is a popular supervised learning algorithm which can be used for both regression and classification. However, it will only be described from a classification perspective since that is what it is used for [11]. The SVM algorithm considers each training instance as a data point in a k dimensional space where k is the number of features and attempts to classify these data points using a k-1 dimensional hyperplane. There can be several hyperplanes through which the data points are separable. However, the hyperplane for which the distance of nearest data point of each class from itself is minimum. This distance is also known as the margin and the greater is the margin, the lower is the chance of error while classifying data at the inference time. In case, a perfect separation between the data points is not possible, the hyperplane which results in the best separation is chosen by considering some of the data points as outliers [12].

It can often happen that a linear decision boundary between the data points is not possible given the nature of data. In this case, the SVM algorithm uses a technique known as kernel trick. This mechanism transforms the data to a higher dimensional space by introducing new features where these points can be easily separated. The transformations that a SVM uses to map the feature space is are designed such that the dot products obtained from the pairs of vectors representing the data points in the input space can be easily computed in terms of the variables belonging to the real input space by defining them through a kernel function *k(x,y)* which is well-suited according to the problem at hand [21].

2.5.3 Decision Tree

Decision Tree algorithm is one of the most popular supervised learning algorithms that is used in predictive analytics. It can be used for both classification and regression problems but in our use case, it was used for classification so it's only the methodology used by it for dealing with classification problems will be discussed subsequently. In the classification problems, the target variable that a decision tree attempts to predict is categorical. A decision tree classifier works by splitting the dataset based on the value of an attribute or feature of the data at every subsequent level [30]. A decision tree can split the data into two or more sets but nearly all the commonly used implementations construct decision trees as binary trees where each node can have only two branches. Some of the common terminologies used in a decision tree are as follow:

- **Root Node**: It represents all the instances in the dataset. It performs the first split on the data where it is divided into two or more sets based on some attribute.

- **Leaf Node:** The leaf node in a decision tree determines the class to which a particular instance belongs. A leaf node is reached when the subset formed by a split has instances belonging to a single class. Such a split is known as a pure split.
- **Decision Node**: All the sub-nodes of the root node which perform splits on data based on the data attributed are called decision node. A decision node depicts the fact that the subset of training examples resulting from the split corresponding to a branch of its parent node were associated to more than one classes thereby giving rise to an impure split.

At the training time, the decision tree is constructed using the attributes resulting in best splits at various levels. At the inference time, the given example traverses the tree following the rules dictated by the attribute values at every node unless it reaches a decision node. The choice of the attribute at every node except for the leaf node is made using some statistical measure and a well-defined approach. The most used algorithm for determining the attribute for a split at a given stage is the ID3 algorithm which stands for **Iterative Dichotomiser 3 [42]**. This algorithm uses a greedy approach by choosing the attribute at each step which gives the best split in context of that split only from the set of attributes that have not been chosen yet. This process is repeated unless we reach a leaf node. The statistical measures that are generally used to determine the purity or goodness of a split are as follows:

- **Entropy:** Entropy measures the randomness in the dataset. A greater value of entropy indicates more randomness which makes the splits worse and vice versa. A branch having a 0 entropy is a leaf node and a branch with an entropy greater than 0 is a decision node. Entropy is computed as follows:

$$E(S) = -\sum_{i=1}^{c} (p_i \log p_i) \tag{2.18}$$

Here c is the number of categories in the target variable, S is the subset of the dataset in the given branch of a split and p_i is the percentage of examples or instances belonging to each class in that split. Entropy for multiple attributes is given as follows:

$$E(T, X) = -\sum_{c \in X} P(c)E(c) \tag{2.19}$$

Where X is a given attribute, c are the unique categories in that attribute and T is the target variable.

- **Information Gain:** Information Gain is a property that defines the goodness of an attribute in determining the classification of the output variable. At each level, the ID3 algorithm chooses the attribute that results in the highest information gain. Information gain can also be defined as the loss in entropy. It is computed as follows:

$$\text{Information Gain(T, X)} = E(S) - \sum_{i=1}^{K} E(i, X) \tag{2.20}$$

Where E(S) depicts the entropy of the dataset before the split, K is the number of subsets produced by the split.

- **Gini Index:** Gini Index is a metric that can be used interchangeably with entropy. Its advantage is that it is computationally cheaper than the latter. It is also known as Gini Impurity. It is computed by subtracting the sum of square of percentage or probability of each target class from one. The formula is stated as follows:

$$\text{Gini Index} = 1 - \sum_{i=1}^{c} (p_i)^2 \tag{2.21}$$

The index has a value that varies between 0 and 1. The value of 0 indicates purest classification or division where all instances belong to a single class and a value of 1 indicates random distribution of the instances across all the target classes. One unique quality of this metric is that it favors large partitions.

2.5.4 K-Nearest Neighbors

The KNN algorithm falls in the category of unsupervised machine learning algorithms. It is used for solving both classification and regression related problems. The unique fact about this algorithm is that it does not train on the data or take time to learn from it. It just uses the data to perform inference on a provided example at test time using certain statistical measures. Therefore, it is very computationally efficient in terms of runtime. All the instances of data are considered as data points in a multi-dimensional plane where the number of features equates the number of dimensions, and a test example is plotted in the same plane [8]. The value of K determines the number of neighbors of the data point corresponding to the test example whose vote will be used to classify the data point. The class for which the highest votes are received is deemed as the class to which the data point belongs.

Initially the entire set of training data is loaded into the memory and appropriate feature representations are used to model it is a data point. For any given test instance at runtime, the K most similar or closest data points to it are chosen. Several similarity measures are used. Two of the most popular similarity measures are defined as follows:

- **Euclidian Distance:** Euclidian Distance is defined as the length of the straight line between two data points. It is computed as follows:

$$L(p,q) = \sqrt{\sum_{i=1}^{n} (p_i - q_i)^2} \tag{2.22}$$

Where n is the number of dimensions in the feature space.

- **Cosine Similarity:** Cosine similarity is used to measure the similarity between two vectors using the cosine angle between them. It is computed as follows:

$$\cos(p,q) = \frac{p.q}{|p||q|} \tag{2.23}$$

To choose the K most similar data points, the entire set of data points that have been already classified are sorted in terms of similarity with the test point and top K data points are considered. The most frequent class from amongst these data points is the one to which the test instance belongs.

- **Manhattan Distance:** The Manhattan distance between two points p and q is given as**:**

$$M(p,q) = \sum_{i=1}^{n} (p_i - q_i) \tag{2.24}$$

Where n is the number of dimensions in the feature space.

2.6 Fake News Detection Relevant Literature

A totally accurate, comprehensive, and fully automatic "fake news" method to machine learning is challenging to build [45]. Kouzy [26]. examine the extent of COVID-19 misinformation propagation on Twitter. The study conducted using 14 different trending hashtags and keywords related to the COVID-19 epidemic in order to identify and assess the nature of misinformation tweets. They found out that health misinformation and unverifiable tweet content regarding the COVID-19 pandemic are being disseminated at higher rates on social media. Around 70 per cent of the tweets were about health information, while the others were about sociopolitical and financial factors. Altogether out of 153 tweets (24.8 per cent) included misinformation, and 107 (17.4 per cent) included unverifiable information related to COVID-19. These statistics in this study indicate the importance of finding solutions to tackle health misinformation on Twitter as this is one of the primary goals of this thesis.

Brennen [6] classify some of the main types, sources, and claims of COVID-19 misinformation. They investigate 225 pieces of misinformation rated false or misleading by fact-checkers websites and published in English between January and the end of March 2020. They found out that fact-checkers websites have increased more than 900 per cent to respond to COVID-19 misinformation. Additionally, COVID-19 fake news tweets tend to have some kind of reconfiguration, where true information is twisted and

recontextualised. Most of the misinformation in their study came from normal people. The claims identified in the study were misleading or incorrect claims about public authorities such as governments and the World Health Organisation (WHO). Another important observation in this study is that of all social media platforms turns out that Twitter has 59 per cent of fake news posts which is considered a huge rate.

Cui and Lee [7]. present CoAID which is a dataset that contains data related to COVID-19 healthcare misinformation. It is a labelled fake news dataset that includes 4,251 news, 296,000 related user engagements, 926 social platform posts about COVID-19. The dataset is provided to assist researchers in investigating the COVID-19 health misinformation topic. In their work, they have compared the unique features between misinformation and facts about COVID-19 and ran some state-of-the-art misinformation detection methods on the CoAID dataset. In their dataset, they have collected all possible features of interests, including multimodal information and user engagement data. The methods used in their work are as follows:

- SVM (Support Vector Machine): This method includes bag-of-words text representation. The text representations are put into a linear kernel SVM.
- LR (Logistic Regression): They have concatenated all the word embeddings and fed them to the algorithm. RF (Random Forest): This method also includes bag-of-words text representation.
- Convolutional Neural Network (CNN): In this deep learning algorithm the word embeddings are fed into a Convolution1D to learn filters.
- Bidirectional GRU: This method takes words annotations by summarizing information from directions for words
- CSI: It is deep learning-based misinformation detection model that uses information from news article content and user response. LSTM model is used to represent articles vectors.
- SAME: This model uses multimodal data and users' sentiments for fake news detection
- HAN: This method applies two levels of attention mechanisms at word and sentence level to classify more and less meaningful content in a document.
- dEFEND: This method depends on using a hierarchical attention network and a co-attention mechanism connecting article content and user comment to identify misinformation.

It has shown in their work that state-of-the-art methods perform better than simple methods based on the metrics PR-AUC, Precision, Recall, and F1 score.

Han, Karunasekeran and Leckie [18]. utilise the use of the state-of-the-art method Graph Neural Network (GNNs) to differentiate between the widespread patterns of fake and real news on social media. They have applied this method on two datasets with thousands of labelled news items. This approach of using GNNs does not depend on any text information such as tweet content, replies and user descriptions. Based on the results of their study they found out that Graph Neural Networks (GNNs) achieves excellent performance without relying on any text information. Additionally, GNNs does not perform well on unseen data even with retraining the model on the entire data from scratch. They solved this problem by using an alternative method that uses techniques from continual learning to train the GNNs model and it achieves equivalent performance on both existing and unseen datasets.

Zhou and Zafarani [47]. conduct a comprehensive survey that analyses and evaluates methods that can detect fake news on social media platforms. The false information, its writing form, its propagation patterns, and the reliability of its source. They have also classified related key theories across different disciplines to support interdisciplinary research on fake news issue. They propose fake news style-based method tend to assess news intention such as whether there is an intention to deceive the public or not. Text style is represented by a set of quantifiable characteristics usually ML features. These features can be grouped into textual features and visual features representing news text and images as well.

Wang [44] address the issue of identifying the background knowledge hidden in the text content of the post online which is an important factor in assessing fake news detection. They use a novel Knowledge-driven Multimodal Graph Convolutional Network (KMGCN) to model the semantic representations. This can be done by modelling text information as well as knowledge concepts and visual information. This

approach converts word sequences into a graph, to understand the semantic and all information related to the text. They have also converted visual information as nodes of graphs and retrieved external data from real-world knowledge graph as nodes of graphs to provide complementary semantics information to improve fake news detection. This thesis will follow a similar approach where data represented in a graph consisting of nodes, edges and features. In their work, they modelled multimodal content as graphs including text, visual information to capture non-consecutive semantic features and make use of external knowledge data.

Kim [23] implemented a more efficient approach to detect emerging COVID-19 misinformation. This is done by tracking replies that seem to give correct information. This way of social network analysis shows the extent to which misinformation has broadcasted on social networks. They represented their data in a graph where it has a parent node (post) that is connected to other nodes in the network that gives much information to assess the post information credibility. One of the nodes is volunteer fact-checkers that identify misinformation in the post. Once misinformation is detected in the post, further tracking for misinformation is done on the upstream (Friends nodes) and downstream (Followers Nodes). This methodology overcomes the challenge of discovering random misinformation to that of identifying fact-checked reply posts containing accurate information.

Another study [16] implements a combination of methods to accurately detect fake news in social media. The goal of the proposed method described in the research paper is to have a robust and accurate fake news detection algorithm. The methods used are Linguistic Cue and Network Analysis as well as supervised machine learning techniques including Naïve Bayes classifier, SVM, and semantic analysis. One of the advantages of the Network Analysis approach is that it is a content-based approach that relies on deceptive language cues to predict deception and this is valuable to accurately detect false information.

They produced a study in which two main experiments were performed, text-based and graph structure based fake news detection. Their main aim was to analyze 5G conspiracy COVID-19 tweets and COVID-19 tweets to detect spreading of misinformation. In the first experiment, BoW and BERT methods were used where Naive Bayes classifier was used for the former and logistic regression for the latter. For the second experiment, graph neural networks were used in this study. This proved to one of the major advantages since deep learning methods are known to generate improved results when compared to machine learning algorithms. Hence, their results were highly accurate and can be state-of-the-art.

2.7 Dataset

2.7.1 Introduction

There are 7 datasets stored in JSON format used in the project implementation listed as the following:

- covid-misinfo-terms- small.json (Misinformation Tweets)
- scamdemic.json (Misinformation Tweets)
- plandemic.json (Misinformation Tweets)
- smaller-coronavirus0.json (Non-Misinformation Tweets)
- smaller-coronavirus1.json (Non-Misinformation Tweets)
- smaller-coronavirus2.json (Non-Misinformation Tweets)
- smaller-coronavirus3.json (Non-Misinformation Tweets)

These COVID-19 tweet records have been cleaned, labelled and converted to separate CSV files suitable for each experiment.

2.7.2 Data Source

The data were collected by Dr.Ryan McConville back in march 2021. The hashtag keywords in the query script that were used to extract the misinformation tweets are the following:

- plandemic
- scamdemic

- wuhanvirus
- chinavirus

The hashtag keyword in the query script that was used to extract the non-misinformation tweets is the following:

- coronavirus

2.7.3 Data Overview

The final dataset contains 322683 records. The important features selected for the training process are the following:

- **Created at**: The UTC time when a tweet was created.
- **Id**: A unique identifier for a particular tweet.
- **Conversation id**: An identifier on each reply, which matches the tweet ID of the original tweet that started the conversation.
- **Author id**: An identifier for the user who posted the tweet.
- **Text**: The plain text of the tweet.
- **In reply to user id**: An integer representation of the original tweet's author ID.
- **Referenced tweets**: A nested object that has retweeted/replies for a tweet ID.
- **entities.mentions**: An object includes mentions to other usernames.

Chapter 3

Project Implementation

3.1 Introduction

This chapter discusses the implementation aspect of the proposed fake information detection system. Furthermore, this study implements various different machine learning algorithms along with multiple features type to automatically detect false information. In order to identify fake tweets automatically, this study concentrates on various text features, such as BoW, TF-IDF, Word2Vec. These features act as input to different classifiers. These classification algorithms are the following:

(i) Multinomial Naive Bayes (MNB)
(ii) Gaussian Naive Bayes (GNB)
(iii) Decision Tree Classifier (DT)
(iv) K-Nearest Neighbor (KNN)
(v) Support Vector Machine (SVM)

This study also evaluates the performance of each classification algorithms using different metrics, such as precision, recall, accuracy, ROC and f1-score.

3.2 Project Management

I used git version control system on my local machine to keep track of project experiments code as well as the data files.

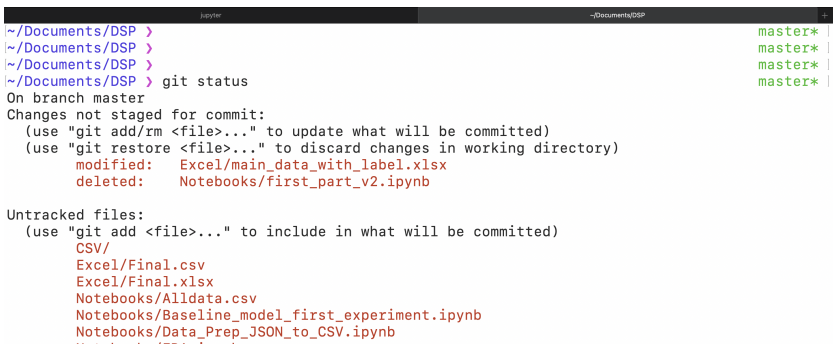

Figure 3.1: A screenshot showing the repository status on my local machine

```
jupyter                                                                    git
81f91a5 (HEAD -> master) Add accuracy and classification report evaluation metrics
5b9c78d Add label to tweet text based on certain keywords
dcce06f Add changes to build supervised ML classification models
d38fe25 Add main data excel file with label
f024198 Change notebook name to first_part
ee041b4 Add reading data and first part notebooks
23e48f8 Add data files in excel format
eaa71ef Add gitignore file
c0d3aed Add data files in json format
(END)
```

Figure 3.2: A screenshot showing the repository log on my local machine

3.3 Dataset Creation

```
IMPORT json, pandas
IMPORT urlopen FROM urllib.request
BIGEN
AllDataPd ← [ ]
Procedure Normalize_JSon_File
for each Filename do
    File ← READ(Filename)
    Data ← LOAD(Filename)
    DataPd ← JSON_NORMALIZE(Data['tweets'])
    APPEND(AllDataPd, DataPd)
    CLOSE(File)
end for
Procedure Extract_Nested_JSON_Objects
Referenced Tweets ← r['referenced_tweets']
ArrReferencedTweets ← empty dictionary
if IsINSTANCE(ReferencedTweets,LIST) then
    DataReferencedTweets ← JSON_NORMALIZE(ReferencedTweets)
end if
for (i,r) IN ITERATIONROWS(DataReferencedTweets) do
    ArrReferencedTweets['TYPE(ReferencedTweets)' + STRING(i+1)] ← r['type']
    ArrReferencedTweets['ID(ReferencedTweets)' + STRING(i+1)] ← r['id']
end for
APPEND(ReferencedTweets,ArrReferencedTweets)
EntitiesMentions ← r['entities.mentions']
ArrEntitiesMentions ← empty dictionary
if IsINSTANCE(EntitiesMentions, LIST then
    Data Entities Mentions ← JSON_NORMALIZE(EntitiesMentions)
    for (i,r) IN ITERATIONROWS(DataEntitiesMentions) do
        ArrEntitiesMentions['entities.mentions.username' + STRING(i+1)] ← r['username']
    end for
    APPEND(MentionUsername,ArrEntitiesMentions)
    DataMentionUsername ← JSON_NORMALIZE(MentionUsername)
    DataMentionUsername ← JSON_NORMALIZE(ReferencedTweets)
    return  AllDataPd, MentionUsername, ReferencedTweets
end if
END PROCEDURE
Procedure Normalize_JSon_File
TO_CSV(AllDataPd, csv file)
```

Algorithm 3.1: Pseudocode for Parsing and Converting JSON Data Into a CSV Format

The pseudocode for creating the dataset can be seen in Algorithm 3.1. The whole procedure was carried out using json and pandas data-frame. By the end, the CSV file is stored in an empty list created initially and that is to be used further.

3.4 Dataset Overview

The main dataset used in the project implementation contains 237743 records and 28 features. The dataset comprised of nine columns, where first column corresponded the personality type and second column corresponded to the post. Figure 3.3 shows an example of the 10 rows of the dataset. The first column represents the tweet ID, the second column shows the Id of the author who posted the tweets on Twitter. Likewise, the third colum illustrates the device that was used to post twitter posting. Similarly, the fourth column shows the text of the tweet. The fifth column contains one of two labels (i) True or (ii) Fake. The true tweets are annotated with 1, and fake tweets are annotated as 0. The dataset contains 179681 tweets while 58062 tweets are labeled as Fake. The six column contains the number of retweets of a particular tweet. In addition, the seven column represents the number of replies of the tweets while column eights reveals the number of likes. Finally, the last column shows the counts of quotes.

For example, in Figure 3.3 second row, the number 4056 in the six column represents the number of retweets of that tweet. Similarly, the column seven, eight, nine, and 10 represents public reaction in the form of reply, like, and quote. Finally, on average each tweets in both labels (True and Fake) contains approximately 150 words.

	id	author_id	source	text	label	retweet	reply	like	quote
0	1357411153000000000	1289288731000000000	Twitter for Android	Should there even be an All-Star Game this yea...	True	0	0	0	0
1	1357411153000000000	1275531383000000000	Twitter for Android	RT @narendramodi: Spoke to President @CyrilRam...	True	4056	0	0	0
2	1357411152000000000	4825557477	Twitter for iPhone	RT @TorontoStar: How risky are Ontario schools...	True	43	0	0	0
3	1357411152000000000	2866489102	Twitter Web App	RT @dankennedy_nu: Now up at @storybench: How ...	True	1	0	0	0
4	1357411152000000000	22970986	SocialNewsDesk	'Don't treat this lightly': Coronavirus surviv...	True	0	0	0	0
5	1357411152000000000	1211482699000000000	Twitter for iPhone	This is considered a felony with Covid going o...	True	0	0	0	0
6	1357411151000000000	1891191457	Twitter Web App	@SteveLawrence_ You would say that but every v...	True	0	0	0	0
7	1357411151000000000	1293819643000000000	Twitter for iPhone	RT @tiwtter2345: #闫丽梦 \n#LimengYan \n\n#Covid1...	True	2	0	0	0
8	1357411151000000000	2578851859	Twitter for iPhone	RT @HelenaMorenoLA: Last year we didn't know C...	True	1	0	0	0
9	1357411151000000000	103154813	Twitter for iPhone	RT @NoEmmeG: man we could roll out so many suc...	True	23	0	0	0

Figure 3.3: The first 10 rows of the dataset

3.4.1 Data Wrangling

Here I explain the steps taken to prepare the given large scale data into a suitable form for the study analysis. Overall, I read the JSON data files into a large pandas dataframe and then export it into a CSV file. Figure 3.4 shows a sample of the data in JSON format.

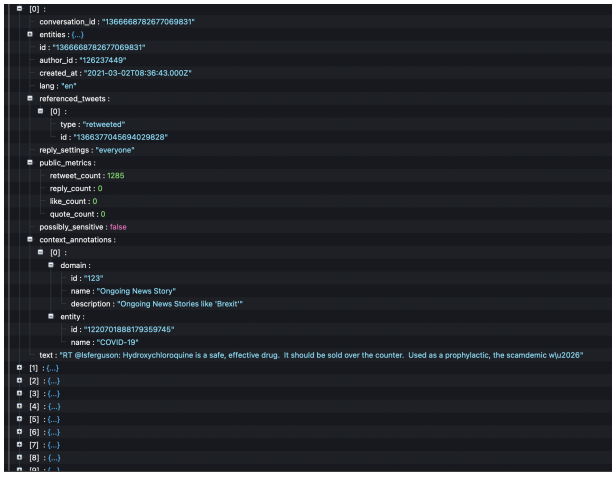

Figure 3.4: Nested JSON Dataset Structure

The steps I implemented are the following:

1. First, I load all JSON data file and appended them into separate lists.

2. Second, I normalized the data by using pandas function json normalize() on “tweets” because it is the head of the JSON data structure.

3. Extracted nested JSON objects to be in seprarate columns in the dataframe . These nested objects includes "referenced tweets" which had the tweet id of retweets/replies to a given tweet. This information is needed later to represent the data in a graph.

4. Third, concatenate all lists into a pandas dataframe.

5. Finally, export the dataframe into a CSV file.

3.4.2 Exploratory Data Analysis

In this part I did some analysis to get more information about the dataset. The analysis conducted are the following:

1. Visualising the label feature count as illustrated in 3.5. Here we have 179681 true tweets and 58062 fake tweets based on the data annotation assumption mentioned in 3.4.3 section. Figure

2. Viewing a sample of the tweet text as illustrated in Figure 3.6.

3. Identifying the length of the tweet text for each class (true, fake)as shown in Figure 3.7

4. Most frequent words in the dataset as shown in Figure 3.8

5. Users mentions in true and fake tweets.

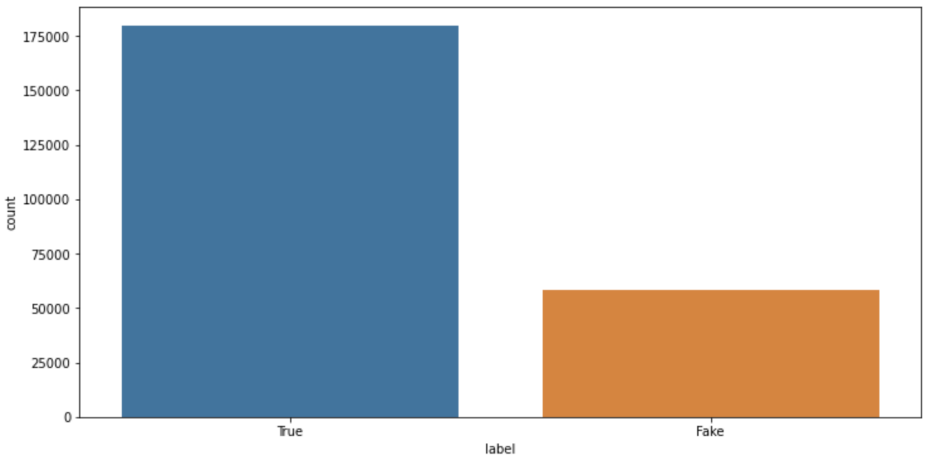

Figure 3.5: A bar chart showing label count

covid
covid19
vaccine
people
new
amp
t
death
coronaviru
government
pandemic
vaccination
relief
school
doctor
american
world
today
make
want
test
uk
free
february
mask
know
need
say
time
year
least
still
state
biden
getting
health
home
day
last
one
rate
case
president
someone
many
public
million
support
due
help
first
think
week
worker
dr
friend
right
s
viru
u
please
florida
number
booked
see
high
data
another
zero
heard
nearly
valenti
higher
update
check
good
thing
month
pay
watch
two
spread
come
may
dont
said
positive
hey
thank
shot
show
side
flu
testing
way
losing
latest
life
democrat
story
look
back
14th
open
must
jaren
start
speech
great
culture
average
got
live
risk
variant
care
student
around
stay
tell
bill
re
dose
countryve
work
sticking
community
face
republican
lot
half
divisive
via
m
c
let
south
moderna
medical
war
read
going
treatment
family
everyone
effective
don
restriction
hospital
yesterday
never
really
china
much
long
well
take
real
grace
emjr
district
taking
lend
india
early
die
billion
house
best
without
minister
made
night
senate
stop
mom
money
chinese
safe
patient
tested
lockdown
black
every
per
whcovidresponse
go
report
other
today
safe
patient
tested
lockdown
black
made
night
restriction
shot
show
side
update
higher
valenti
nearly
heard
zero
check
good
thing
month
pay
watch
two
come
may
dont
said
positive
hey
thank

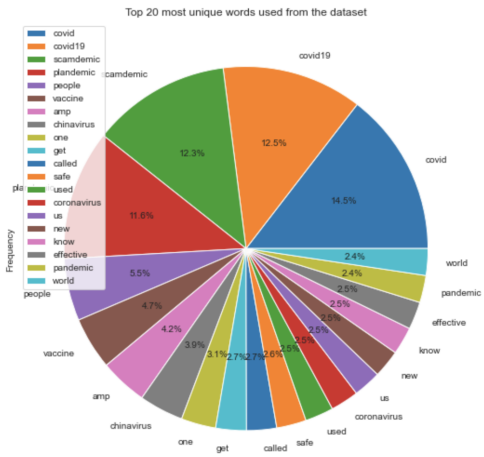

Figure 3.8: Most frequent words in the dataset

3.4.3 Data Annotation

This study uses four hashtag keywords to automatically annotate the complete dataset. The annotation assumption is that if the tweet text contains one of these hashtag keywords it will be labelled as fake otherwise it is labelled as a true tweet. These keywords are the following:

1. plandemic
2. scamdemic
3. wuhanvirus
4. chinavirus

This approach will result in missing information about the label into the features as shown in the experiments results in chapter 4. Therefore, I decided to do an intuitive approach to solve this bias issue which is by removing the set of words from each tweet text that were used as fake labels - only leaving the remaining words. This experiment shows how data collection can influence the model. By removing those words we will remove an artifact of the data collection process.
This dataset contains only tweets in English language. Tweets in Non-English language were discarded. Eventually, the dataset contains 237,743 tweets.

3.5 Architecture of the Experimental Setup

This study split the experimental setup into several stages. The purpose of the research is to experiment with different text features and examining how this affects the performance of the classification models. Hence, I needed to first annotate the tweet text and develop a system for automatically predicting the class (true, fake) of a massive collection of unlabeled tweets. This study will create an informative news corpus that is utilized during the COVID-19 epidemic to analyze the diffusion of false news through social media. Figure 3.9 represents the pipeline that is used in the development of the system to detect false information.

The main implementation steps of the research work are the following:

1. Data Annotation
2. Choosing the important features for the study which is tweet ID, text, label, and referenced tweets IDs.

3. Pre-processing of data to be in the right form to function correctly when feeding it to a classification model.

4. Representing text data using BoW text feature.

5. Representing text data using TF-IDF text feature.

6. Representing text data using Word2Vec text feature.

7. Representing the data in a graph and using Node2Vec algorithm to automatically learn the feature representation.

8. Development of the prediction model.

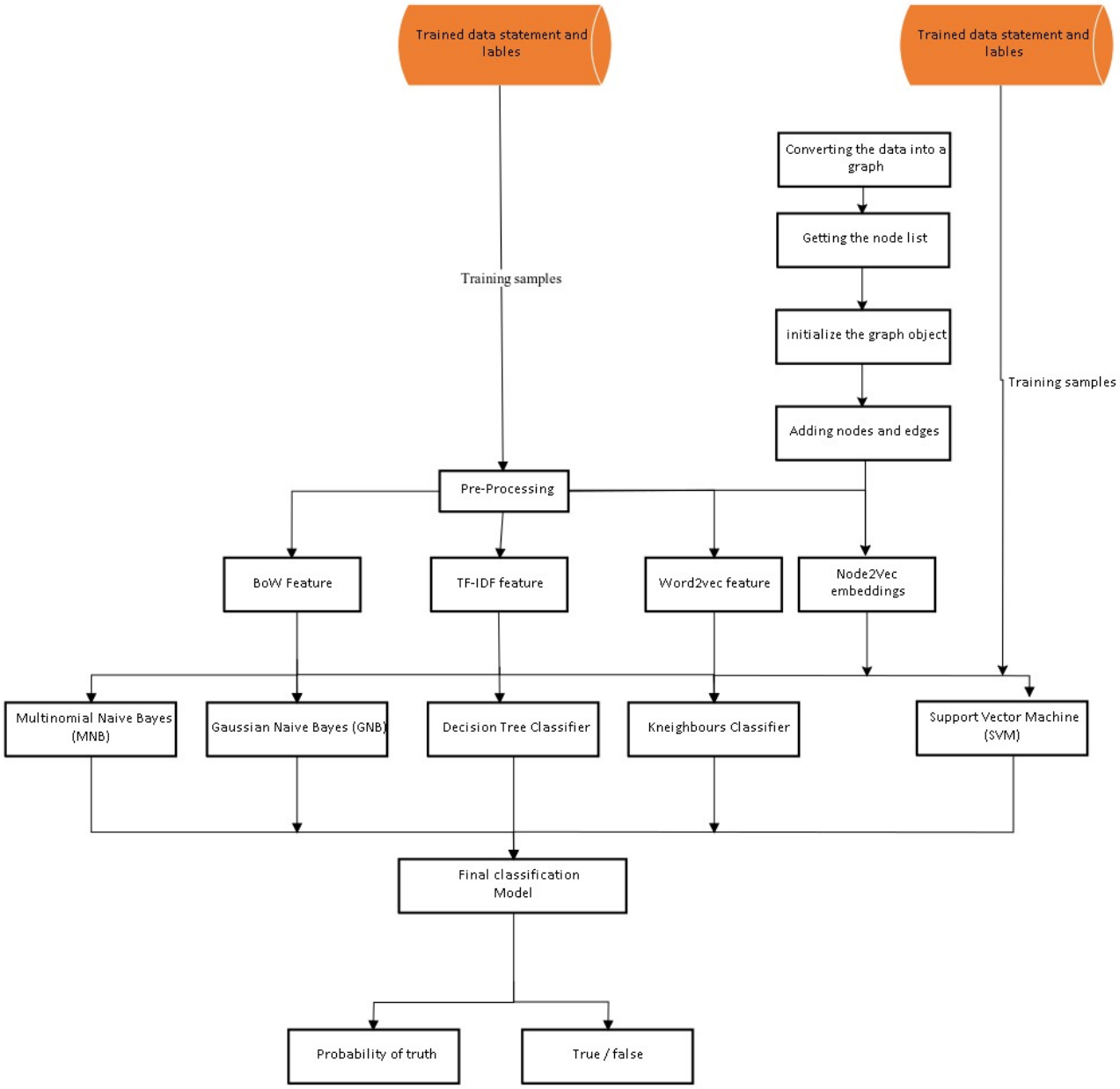

Figure 3.9: The architecture of the proposed pipeline.

3.5.1 Data Pre-processing

Textual data in social media is unstructured; various text analytics have been performed to represent it in a structured fashion. Information retrieval, document classification, topic modelling, and natural language processing are all part of text mining (NLP). NLP enables computers to effectively grasp and analyse human language. NLP's fundamental goal is to extract information from a large text corpus. Pattern matching, normalization, lemmatization, tokenization, tagging, linguistic and lexical analysis are all tasks handled by text pre-processing in natural language processing pipelines. [19]. In this study I used a number of steps to pre-process the data:

- Cleaning up the text contents by lowering case the tweet text.
- Removing retweets tag "RT" from the tweet text as it does not add value to the meaning of the text.
- Removing link website "http" from the text.
- Removing mention symbol (@) from the text.
- Removing hashtag sign from the text.
- Removing punctuation from the text.
- Removing emojies from the text.
- Deleting short words such as be, a, as..etc
- Removing stop words from the tweet text.
- Text tokenisation

3.5.2 Feature Extraction

I conducted experiments to build classification models relying only on one feature which is BoW before focusing on extracting more features to improve the performance. In this study, I chose the following features to experiemnt with:

1. BoW
2. TF-IDF
3. Word2Vec
4. Node2Vec for feature learning on graphs.

For the first part of the study, I simply look at the tweet text to classify tweets. For the second part of the study, I study the effect of graph structure classification where I check the retweets of a given tweet to investigate the dissemination of disinformation. The namely incorrect and partially false tweets and compute the pace of retweets as an agent for the speed of propagation. A retweet is a tweet re-posting, which may be done by or without a remark by a Twitter user [14].

	id	author_id	public_metrics	source	text	label
0	1357411153000000000	1289288731000000000	{'retweet_count': 0, 'reply_count': 0, 'like_c...	Twitter for Android	Should there even be an All-Star Game this yea...	True
1	1357411153000000000	1275531383000000000	{'retweet_count': 4056, 'reply_count': 0, 'lik...	Twitter for Android	RT @narendramodi: Spoke to President @CyrilRam...	True
2	1357411152000000000	4825557477	{'retweet_count': 43, 'reply_count': 0, 'like_...	Twitter for iPhone	RT @TorontoStar: How risky are Ontario schools...	True
3	1357411152000000000	2866489102	{'retweet_count': 1, 'reply_count': 0, 'like_c...	Twitter Web App	RT @dankennedy_nu: Now up at @storybench: How ...	True
4	1357411152000000000	22970986	{'retweet_count': 0, 'reply_count': 0, 'like_c...	SocialNewsDesk	'Don't treat this lightly': Coronavirus surviv...	True
...	...	...	...	...	...	...
237738	1356123683555687936	2816811781	{'retweet_count': 0, 'reply_count': 0, 'like_c...	Twitter Web App	@ADKHospital @HPA_MV @MoHmv Tony Robbins just ...	Fake
237739	1356111948665019904	958710380968034432	{'retweet_count': 0, 'reply_count': 0, 'like_c...	Twitter Web App	@SkyNews Flu has been rebranded as COVID, the ...	Fake
237740	1356110942610857984	958710380968034432	{'retweet_count': 1, 'reply_count': 0, 'like_c...	Twitter Web App	Flu has been rebranded as COVID, the PCR test ...	Fake
237741	1356103385640911104	1349537544610868992	{'retweet_count': 0, 'reply_count': 0, 'like_c...	Twitter for iPhone	Nearly a month on... and this ridiculously acc...	Fake
237742	1356102083049644032	1163040056837325056	{'retweet_count': 0, 'reply_count': 1, 'like_c...	Twitter for Android	Vaccines normally take decades of research, de...	Fake

Figure 3.10: Important features selection.

The following procedures have been used to choose the feature;

1. Writing a BoW encoding function where I tokenize a collection of text documents and build a vocabulary of known words.

2. Splitting the data into train and test sets.

3. Writing a TF-IDF encoding function to apply it on the data.

4. Writing a Word2Vec encoding function to apply it on the data.

5. This information provides the textual materials a considerably more detailed representation.

After the pre-processing and feature extraction phase, I apply each feature on the text and use it as input to a given classifier.

3.6 Text Modelling

In the text modelling part, I have X variable as the tweet text and Y as the label (true, fake). I converted the processed tweet text from dataframe to an array because when splitting the data sets only an array format is accepted in the X variable. After spiltting the data into train and test sets I got the following shape for the train and test sets respectively: ((190194,), (47549,)).
The research experiments are the following:

1. The first experiment, I build DT classifier as a baseline model depending on BoW and TF-IDF feature representation.

2. The second experiment, I try to improve the baseline model by removing the hashtag keywords that were used previously to annotate the data. This is because these keywords have been used to collect and annotate the data and so I need to remove them to improve the model performance.

3. For the third experiment, I conduct independent feature representation with different classifiers as the following:

Classifier	Features
MNB	BoW TF-IDF Word2Vec
GNB	BoW TF-IDF Word2Vec
KNN	BoW TF-IDF Word2Vec
DT	BoW TF-IDF Word2Vec

Table 3.1: The classifiers with various independent feature representation

4. Taking the experiment a step ahead to try different feature combination with each classifier as the following:

   - **BoW and TF-IDF**
   - **BoW and Word2Vec**
   - **TF-IDF and Word2Vec**
   - **BoW, TF-IDF and Word2Vec**

3.7 Graph Modelling

This part of the research involves representing the data in a graph structure and getting the embeddings of the structure using Node2Vec algorithm. The graph consists of tweet id as source nodes and retweets/replies as connections between those nodes (destination nodes). This will enable us to see how many retweets/replies each tweet receive. Hence, classifying the tweet based on its structure not the text itself. Figure 3.11 shows the built graph.

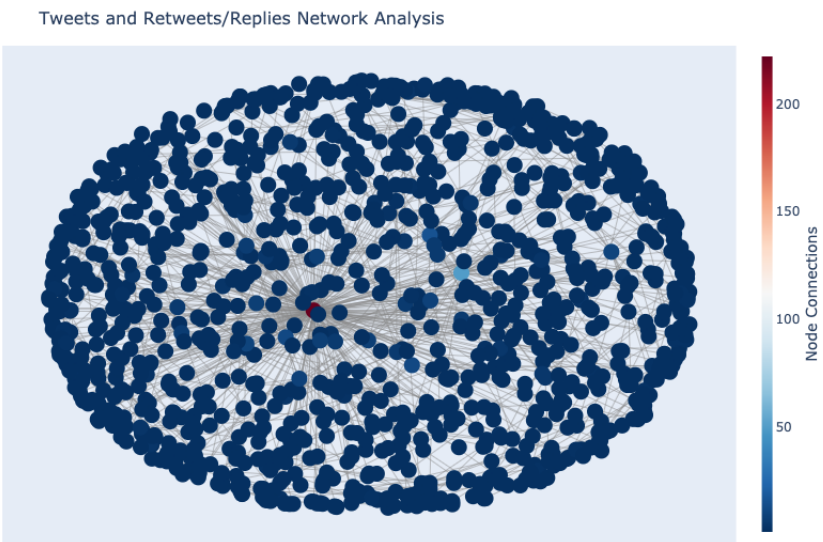

Figure 3.11: A graph representing the tweet ID and its retweets/replies connections

3.7.1 Graph Creation

I read the data into a dataframe. Next, Data was cleaned to remove any missing value tweets samples based on tweet id and retweets/replies column. Then, I made a list of unique nodes to initialize the graph. Each data sample from tweets was added as an edge in the graph. I then got the Node2Vec embeddings and saved it into a CSV file to help us classify the tweets as fake or true. I also stored labels for each node in a separate file.

3.7.2 Embeddings and data labels loading and processing

I read the node embeddings from the saved CSV file in the form of strings. I then splitted each string based on space character and converted them into numeric values so that it can be processed by the classifier. I also loaded the labels file and extracted labels that are associated only with the source nodes which in our case is the tweet (id) column. Node embeddings as "X" and labels as "y" were split into training and testing data of 80% and 20% respectively.

3.7.3 Training and Testing of The classifiers

In the graph modelling part I trained five different ML classifiers including DT, KNN, GNB, MNB and SVM classifiers on the training data. I tested each classifier on the testing data and calculated metrics such as accuracy, precision, recall and F1-Score. I also printed classification report and confusion matrix for each classifier on testing data.

After getting the Node2Vec embeddings I implemented the following scenarios:

- **Node2Vec and MNB classifier**
- **Node2Vec and GNB classifier**
- **Node2Vec and KNN classifier**
- **Node2Vec and DT classifier**
- **Node2Vec and SVM classifier**

The following are defined by these classifiers:

3.7.4 Support Vector Machine (SVM)

It is a supervised model of learning and works in a linear and non-linear classification with high efficiency by use of the kernel technique [9]. It divides the diagrams by separating them using a hyperplane into distinct classes. The dimension of hyperplane changes according to the number of features. As there might be numerous options for a hyperplane to exist in an N-dimensional space, the objective is to discover the plane that separates the data points of two classes with largest margin.

3.7.5 K-Nearest Neighbor (KNN)

In order to increase the overall accuracy for the purposes of classification between a true and a false article we suggested leveraging current ensemble approaches and textual features input [38]. These together are more accurate, because several models are trained to lower the total error rate utilizing a certain approach in order to enhance model performance. The logic underlying the ensemble modelling is identical with that which we have previously experienced in our everyday lives, such asking opinions of several specialists before making a certain decision to decrease the possibility of wrong decisions or of an unwanted result [15].

3.7.6 Decision Tree (DT)

There is a flowchart-like structure in a decision tree. It has three node types, the decision node, the casual node, and the final node. A decision tree discovers specific principles of decision forms the result that the leaf nodes reflect [22]. The decision tree's categorization rules are represented by the roots of the decision tree's leaves. It is extremely helpful in simple jobs in which one can see where to categorize things with a simple logic. It is easy to explain to customer and easy to illustrate how a decision process works, one of the most used algorithms because of its simplicity and usefulness.

3.7.7 Gaussian Naive Bayes (GNB)

This approach works best for continuous data and normal distribution characteristics. Each characteristic is pre-ordained to be autonomous. This model can very simply be taught since it can classify on a little quantity of data during training. A minimal quantity of training data is required for this method to determine the required parameters. In comparison to advanced approaches, Naive Bayes classifiers are incredibly quick [41]. Model Naive Bayes for very big data sets is straightforward to construct and very beneficial.

3.8 Evaluation Methods

Model evaluation is a significant tasks in machine learning. This study demonstrate different evaluation metrics to evaluate the performance of the proposed models using accuracy,precision, recall, and f1-score. The accuracy metric define the number of correctly classified sample by a classifier. The recall metric shows how well the model classifies the positive cases, and precision metric demonstrate how well the model prevents the negative cases classified as positive.

The algorithm's evaluation performance is critical because it figure out that how well the algorithm performs in its classification conjointly makes a distinction in selecting the more precise classifier. The prediction performance of supervised learning models can be evaluate through particular measurements that can best control on with a confusion matrix. For each binary classification task, a confusion matrix compares the predicted actual to the expected classification.

There are four categories of predictions that can be encountered:

- True Positive: Instances that are correctly predicted by the classifier.
- False Positive: Instances that are incorrectly predicted by the classifier.
- True Negative: Instances that are not predicted by the classifier but should be predicted.
- False Negative: Instances that are not predicted by the classifier but should not be predicted.

Table 3.2 illiterates the confusion matrix.

	Actually Positive	**Actually Negative**
Predicted Positive	True Positive (TP)	False Positive (FP)
Predicted Negative	False Negative (FN)	True Negative (TN)

Table 3.2: Confusion matrix.

Precision is defined as the percentage of correct positive predictions out of all positive predictions. False Positives are denoted by the letter FP. Most instances projected as true positives are successfully classified with high precision.

$$\text{Precision} = \frac{TP}{TP + FN}.$$

TP signifies the number of True Positive cases and FN denotes the number of False Negative cases in recall, also known as sensitivity. A high recall indicates that the majority of positive cases could be identified correctly.

$$\text{Recall} = \frac{TP}{TP + FN}$$

Accuracy is the proportion of true results among all instances examined.

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}$$

The f1-score is a single metric that combines precision and recall. Precision and recall are equally weighted in this metric. Similarly, a f1-score reaches its highest value at 1 and its lowest value at 0.

$$\text{F1}_{\text{score}} = 2 \times \frac{precision \times recall}{precision + recall}$$

Since we have an imbalanced dataset, the F1 score is considered a suitable measure for this case. Therefore, it will be the primary metric along with other metrics in this research.
[1]
[2]
[3]

data

[1] Oral Presentation YouTube Link: https://youtu.be/DFNR-0xEzbg
[2] GitHub Repository Link: https://github.com/AlmalkiHub/DSP
[3] Data Files Link on Google Drive: https://drive.google.com/drive/folders/13SLsFCMiw7zdNJx8z01iyX7-9Clfq9ZG?usp=sharing

Chapter 4

Project Results and Critical Evaluation

4.1 Introduction

This chapter describes the experiments taken to establish a classification system and lists the results for each experiment. Additionally, evaluating the significance of the results and overall success of the project.

4.2 Approach to Result Generation

To produce the results, supervised ML classification models were trained with independent and combined text features to test and compare the performance for the following scenarios:

- Text-based classifications models with independent features including BoW, TF-IDF, and Word2Vec embeddings algorithms.
- Text-based classifications models with combined features including the following feature combinations:
  - **BoW and TF-IDF**
  - **BoW and Word2Vec**
  - **TF-IDF and Word2Vec**
  - **BoW, TF-IDF and Word2Vec**
- Graph structure classification using Node2Vec embeddings.

These were chosen in particular to show performance across a range of scenarios to check the performance differences. The metrics used to evaluate the models are Accuracy and F-1 Score. However, since the dataset that we have is not balanced hence the classification accuracy is not a good metric, in this case, is also used alongside other metrics and its implications are known. The results are obtained from the prediction implementation described in Chapter 3. Two main classification techniques are used: text-based and graph-structure. The performance of these methods is compared and evaluated.

4.3 Baseline Model Results

This study used a baseline model to set a minimum threshold to evaluate the performance of the machine learning classifiers. BoW and TF-IDF feature vectors were used to train the DT classifier. The DT algorithm obtained 90.00% Weighted Average F1 Score as shown in Table 4.1.

Classifier	**Fake Class**			**True Class**			**Micro**	**Weighted**	**Accuracy**
	Prec	**Recall**	**F1-Fake**	**Prec**	**Recall**	**F1-True**	**avg F-1**	**avg F-1**	
DT	0.77	0.88	0.82	0.96	0.91	0.93	0.90	0.88	0.90

Table 4.1: DT Classifier - Baseline Model

4.4 Removing Hashtag Keywords Results

In this experiment I tried to improve the baseline model performance. Therefore, I have removed the hashtag keywords including plandemic, scamdemic, wuhanvirus and chinavirus from the tweet text leaving only the remaining words. This is because these keywords were used for labelling the data and is no longer needed to be in the text. The DT algorithm obtained 91.00% Weighted Average F1 Score as shown in Table 4.2.

Classifier	**Fake Class**			**True Class**			**Micro**	**Weighted**	**Accuracy**
	Prec	**Recall**	**F1-Fake**	**Prec**	**Recall**	**F1-True**	**avg F-1**	**avg F-1**	
DT	0.89	0.83	0.86	0.94	0.97	0.95	0.93	0.91	0.93

Table 4.2: After removing hashtag keywords - DT Classifier

4.5 ML Models with Independent Features Results

I have selected three different features to evaluate the results. Those features are BoW-based features, TF-IDF-based features, and Word2Vec-based features. BoW is a way to extract features from text using different ML algorithms. Whereas, TF-IDF is generally used to evaluate the importance of a text in a document. Moreover, Word2Vec is used to victories text. Considering the features, I again classify them into two different classes i-e: true and fake classes. I evaluate results using different performance metrics: precision, recall, and F1 score for both classes. Metrics are commonly used for quantitative comparison of the performance of different classifiers. Five classifiers are used to evaluate the results. If we consider fake classes then MNB, GNB, DT, KNN and SVM were run and computed its accuracy against each feature. I also evaluate all the features based on both true and fake classes.

On computing accuracy and ROC, I see that when we compute the accuracy of BoW-based features I get different accuracies scores for different classifiers. Like, we get an accuracy of 0.76 for the MNB classifier. GNB gives an accuracy of 0.60. While evaluating results using the DT classifier, we get 0.47 accurate results. Whereas, KNN accurately predict 0.71 features as shown in Table 4.2. These are the results after calculating all the performance metrics and finally get accuracy. If we compare the classifiers, we can see that MNB has high accuracy score in predicting data objects. Now if we compare the ROC value of each classifier against each class, we again get different values. Some of the classifiers have high ROC whereas some of the classifiers have low ROC values. ROC is used to compare sensitivity against specificity across a wide range of values. These values of accuracy and ROC show that how classifiers can perform using classes and multiple features. While comparing the ROC of classifiers we can see that MNB classifier has a high ROC value of 0.71 as compared to others.

Classifiers	**Fake Class**			**True Class**			**ROC**	**Acc**	**Micro**	**Weighted**
	Prec	**Recall**	**F1-Fake**	**Prec**	**Recall**	**F1-True**			**avg F-1**	**avg F-1**
MNB	**0.53**	0.35	**0.42**	**0.81**	0.90	**0.85**	**0.71**	**0.76**	**0.64**	**0.75**
GNB	0.32	**0.54**	0.40	0.81	0.62	0.70	0.62	0.60	0.55	0.63
DT	0.19	0.09	0.12	0.75	0.88	0.81	0.42	0.47	0.46	0.64
KNN	0.19	0.05	0.08	0.75	**0.94**	0.83	0.46	0.71	0.45	0.65

Table 4.3: BoW-based feature with various classifiers

Regarding the results of TF-IDF-based features, again all the classifiers were run and then noted the accuracy. MNB classifier gave 0.77 accurately predicted objects. Whereas, GNB, DT, and KNN classifiers have 0.63, 0.69, and 0.70 of accuracy respectively as shown in Table 4.4. While talking about ROC value, MNB have a high ROC value of 0.69 while other classifier does not perform well using TF-IDF-based features. If we compare these classifiers based on their performance, then we can say that the MNB classifier has better performance while considering TF-IDF-based features.

Classifiers	Fake Class			True Class			ROC	Acc	Micro	Weighted
	Prec	Recall	F1-Fake	Prec	Recall	F1-True			avg F-1	avg F-1
MNB	**0.64**	0.17	0.27	0.78	**0.97**	**0.87**	**0.69**	**0.77**	**0.57**	**0.72**
GNB	0.32	**0.42**	**0.36**	**0.79**	0.71	0.75	0.55	0.63	0.55	0.65
DT	0.19	0.07	0.11	0.75	0.90	0.82	0.46	0.69	0.46	0.64
KNN	0.18	0.05	0.08	0.75	0.92	0.83	0.55	0.70	0.45	0.65

Table 4.4: TF-IDF based feature with various classifiers

Regarding Word2Vec-based features, if we evaluate the results with having an accuracy of 0.75 for MNB, 0.80 for GNB, 0.59 for DT, and 0.73 for KNN as shown in Table 4.5. We also get ROC results having values of 0.80 for MNB, 0.81 for GNB, 0.59 for DT, and 0.73 for KNN. If I compare the results of classifiers and their performance by using different performance metrics, I concluded that GNB classifier has a better performance for Word2Vec features by considering both the accuracy and ROC metrics.

Classifiers	Fake Class			True Class			ROC	Acc	Micro	Weighted
	Prec	Recall	F1-Fake	Prec	Recall	F1-True			avg F-1	avg F-1
MNB	0.00	0.00	0.00	0.76	**1.00**	0.86	0.80	0.75	0.43	0.65
GNB	**0.63**	0.46	**0.53**	**0.84**	0.91	**0.88**	**0.81**	**0.80**	**0.70**	**0.79**
DT	0.30	**0.51**	0.38	0.80	0.62	0.70	0.53	0.59	0.54	0.62
KNN	0.45	0.47	0.46	0.83	0.81	0.82	0.66	0.73	0.64	0.73

Table 4.5: Word2Vec-based feature with various classifiers

These tables 4.2 4.4 4.5 show the detailed comparison of performances of the given classifiers. Lastly, if we compare three features along with both classes as true and fake and want to compute the results as a whole on given classifiers, we can see that from the results classifiers perform better with Word2Vec based features. This is because all of the performance metrics have high throughput showing that classifiers have a better prediction of classes with having Word2Vec features. This is because Word2Vec features vectorises the text objects for better classification and prediction purposes. Moreover, Word2Vec was the most accurate out of all independent features as shown in Figure 4.1

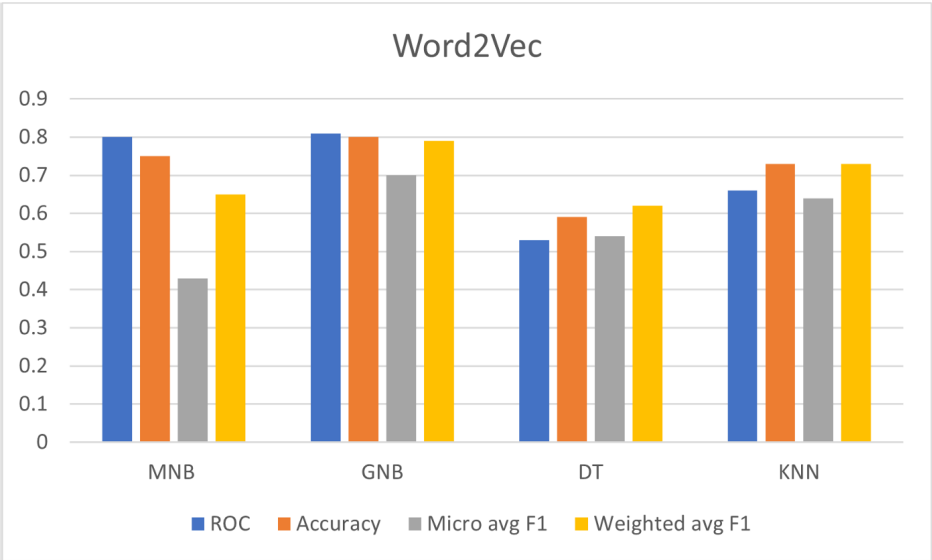

Figure 4.1: Usage of Word2Vec features generates the most accurate results

4.6 ML Models with Combined Features Results

Here I have decided to move from single features to a combination of features. I run the classifiers on the combination of features like first, I decided to combine BoW and TF-IDF. Then, BoW was combined with Word2Vec, TF-IDF and Word2Vec was combined, lastly, BoW was combined with TF-IDF and Word2Vec. After the combination, five different classifiers were selected to evaluate the results for classification. I again here classify the dataset into two classes as true and fake. The performance of these methods is compared and evaluated using multiple metrics including precision, recall, F1-score, and ROC measures. After measuring performance metrics we then calculate the accuracy of the classifiers for accurately classifying text data into its respective class.

The first features were combined and computed results using SVM classifier as shown in 4.6. The accuracy of the classifier in classification is 0.83 and has an ROC of 0.83 for the (BoW + TF-IDF) combination. Again, SVM was run with a combination of (BoW + Word2Vec) and got an accuracy of 0.83. From the above-generated results by the classifiers, we can conclude that SVM performs better with features combination.

Again, all of the feature combinations are given to the classifiers as shown in Tables 4.6, 4.7, 4.8, 4.9 and 4.10 for evaluating the results and make a comparison between all the classifiers and show which of the classier has a high true positive rate. If we compare all the results, we can conclude with the fact that despite the combination of the feature we still have the classifier SVM which can perform better in this case as well. As seen from the above-mentioned tables we can evaluate the results and make a statement that the SVM classifier has high accuracy which means that SVM classifies the test dataset accurately having a high true positive rate.

Classifiers	**Fake Class**			**True Class**			**ROC**	**Acc**	**Micro**	**Weighted**
	Prec	**Recall**	**F1-Fake**	**Prec**	**Recall**	**F1-True**			**avg F-1**	**avg F-1**
BoW + TF-IDF + SVM	0.75	0.52	0.61	0.86	0.94	0.90	0.83	0.83	0.76	0.83
BoW + Word2Vec + SVM	0.75	0.52	0.61	0.86	0.94	0.90	0.83	0.83	0.76	0.83

Table 4.6: Combined features with SVM classifier

Classifiers	**Fake Class**			**True Class**			**ROC**	**Acc**	**Micro**	**Weighted**
	Prec	**Recall**	**F1-Fake**	**Prec**	**Recall**	**F1-True**			**avg F-1**	**avg F-1**
BoW + TF-IDF + MNB	0.00	0.00	0.00	0.76	1.00	0.86	**0.86**	0.76	0.43	0.65
BoW + Word2Vec +MNB	0.00	0.00	0.00	0.76	1.00	0.86	0.68	0.76	0.43	0.65
TF-IDF + Word2Vec +MNB	0.00	0.00	0.00	0.76	1.00	0.86	0.75	0.76	0.43	0.65
BoW + TF-IDF + Word2Vec + MNB	0.00	0.00	0.00	0.76	1.00	0.86	0.80	0.76	0.43	0.65

Table 4.7: Combined features with MNB classifier

Classifiers	Fake Class			True Class			ROC	Acc	Micro	Weighted
	Prec	Recall	F1-Fake	Prec	Recall	F1-True			avg F-1	avg F-1
BoW + TF-IDF + GNB	0.69	**0.43**	**0.53**	**0.84**	0.94	0.88	**0.86**	0.81	**0.71**	**0.80**
BoW + Word2Vec + GNB	**0.78**	0.33	0.47	0.82	**0.97**	**0.89**	0.80	0.81	0.68	0.79
TF-IDF + Word2Vec + GNB	0.76	0.34	0.47	0.82	**0.97**	**0.89**	0.82	0.81	0.68	0.78
BoW + TF-IDF + Word2Vec + GNB	0.71	0.38	**0.49**	0.83	0.95	0.88	0.81	0.81	0.69	0.79

Table 4.8: Combined features with GNB classifier

Classifiers	Fake Class			True Class			ROC	Acc	Micro	Weighted
	Prec	Recall	F1-Fake	Prec	Recall	F1-True			avg F-1	avg F-1
BoW + TF-IDF + DT	**0.55**	**0.66**	**0.60**	**0.88**	0.83	**0.85**	**0.76**	**0.78**	**0.73**	**0.79**
BoW + Word2Vec + DT	0.29	0.12	0.18	0.76	**0.90**	0.83	0.51	0.71	0.50	0.67
TF-IDF + Word2Vec + DT	0.38	0.20	0.26	0.78	**0.90**	0.83	0.54	0.72	0.55	0.69
BoW + TF-idf + Word2Vec + DT	0.46	0.32	0.38	0.80	0.88	0.84	0.59	0.74	0.61	0.72

Table 4.9: Combined features with DT classifier

Classifiers	Fake Class			True Class			ROC	Acc	Micro	Weighted
	Prec	Recall	F1-Fake	Prec	Recall	F1-True			avg F-1	avg F-1
BoW + TF-IDF + KNN	**0.63**	**0.67**	**0.65**	**0.89**	0.87	**0.88**	**0.80**	**0.82**	**0.77**	**0.82**
BoW + Word2Vec + KNN	0.37	0.10	0.16	0.76	**0.94**	0.84	0.59	0.73	0.50	0.68
TF-IDF + Word2Vec + KNN	0.35	0.11	0.17	0.76	**0.94**	0.84	0.61	0.73	0.50	0.68
BoW + TF-IDF + Word2Vec + KNN	0.35	0.10	0.16	0.76	0.94	0.84	0.60	0.73	0.50	0.68

Table 4.10: Combined features with KNN classifier

4.7 ML Models with Node2Vec Feature Representation Results

This section is for graph modelling part results. DT and KNN classifiers have test accuracy of 0.55 and 0.59 F1-score of fake tweets for GNB, MNB and SVM classifier is zero for fake tweet prediction as shown in Table 4.11 . In this case, my interpretation for this is that it indicates that the default parameters of the classifier are not best for fitting the classifier and due to a zero F1-score for a prediction a warning is displayed. The warning message is not appearing for DT and KNN classified because the F1-score is not zero for a prediction in those classifiers.

Classifiers	Fake Class			True Class			Acc	Micro avg F-1	Weighted
	Prec	Recall	F1-Fake	Prec	Recall	F1-True			avg F-1
MNB	0.00	0.00	0.00	0.67	**1.00**	**0.80**	**0.67**	0.40	0.54
GNB	0.00	0.00	0.00	0.67	**1.00**	**0.80**	**0.67**	0.40	0.54
DT	**0.33**	**0.36**	**0.34**	0.67	0.64	0.66	0.55	**0.50**	0.55
KNN	**0.33**	0.25	0.28	0.67	0.75	0.71	0.59	**0.50**	**0.57**
SVM	0.00	0.00	0.00	0.67	**1.00**	**0.80**	**0.67**	0.40	0.54

Table 4.11: Node2Vec-based feature with various classifiers

4.8 Results Analysis

In this section, I present the interpretation of the results of the experiments. Moreover, comparisons between different features and classifiers are addressed.

4.8.1 Baseline Model

The baseline model utilized Decision Tree, as shown in Table 4.1, and obtained an average F1 score of 0.90 and an accuracy of 0.90. When the model was used after removing hashtag keywords, both the average F1 score and accuracy increased to 0.93. For fake classes, the precision, recall and F1- Fake score increased from 0.77, 0.88 and 0.82 to 0.89, 0.83 and 0.86 respectively. When true classes were considered, the recall and f1-true scores increased from 0.91 and 0.93 to 0.97 and 0.95 however precision witnessed a slight decrease from 0.96 to 0.94

4.8.2 ML Models with independent and Combined Features

Independent features

When machine learning models were trained using independent features, the use of Word2Vec-based feature ,Table 4.5,generated the most accurate results. On average, Word2Vec obtained an ROC score of 0.70, an accuracy of 00.72, a micro F1 score of 0.58 and a weighted average F-1 score of 0.70. Quantitatively, these are approximately 25%, 2.86%, 13.73% and 4.48% higher than average scores of TF-IDF based features, Table 4.4, and approximately 27%, 12.5%, 9.43% and 4.48% greater than average score of BoW-based features, Table 4.3.

In terms of machine learning models, Whilst using independent features, MNB performed the best out of all 5 machine learning models with BoW and TF-IDF based. When the former feature was used, MNB obtained a 26.7%, 61.7%, 7.04% higher accuracy when compared to GNB, DT and KNN, respectively. In terms of average F-1 score, MNB obtained a score of 0.64 which is 16.36%, 39%, 42.2% improved in comparison to GNB, DT and KNN respectively. Furthermore, MNB outperformed in all other metrics other than recall.

When TF-IDF based features were used, MNB outperformed the other machine learning models in terms of ROC score, accuracy and f1 score. It also performed the best in terms of precision in fake class and recall as well as the F1 score in the true class. Its performance lagged slightly in terms of recall and f1 score in fake classes and precision in true classes however these can be compensated for by its performance in other metrics.

Lastly, when Word2vec-based features were used the results differentiated from the previous experiments. In this scenario, GNB outperformed the other models in all evaluation metrics other than recall. An ROC score of 0.81 was obtained, 1.25%, 50.9% and 22.7% higher than those of MNB, DT and KNN, respectively. In addition to this, a higher accuracy was achieved, of 0.80, by approximately 6.67%, 35.6% and 9.59% when compared to MNB, DT and KNN.

Combined features

For text-based experiments, SVM performed the best when using BoW with TF-IDF and BoW with Word2Vec features in comparison to all other models. It obtained an average ROC score and accuracy of 0.83, and an average F1 score of 0.76 was achieved which proved to be approximately 1.21%, 2.47% and 10.1% higher,respectively, than the second best performing model. However, when all combinations were compared with each other, GNB proved to be the best performing.It obtained an average ROC score of 0.82, 6.49% higher than MNB, 36.67% higher than DT and 26.15% higher than KNN. In this manner, GNB achieved substantially higher cores across the board for different evaluation metrics. It can be justly hypothesized that if SVM were to be used for these other experiments, it would outperform other models as well.

Once the best performing classifier was determined, another revelation made was that a single combination was not optimal across the board when GNB was used. Though using BoW with TF-IDF did achieve high scores for ROC, F1 score, Recall in fake classes, precision in true classes and recall in true classes, using BoW with Word2Vec obtained higher scores in precision in fake classes, recall in true classes and f1 score in true classes. The accuracy remained the same for all combinations, 0.81.

Hence, after analyzing the results quantitatively it can safely be concluded that using a combination of BoW with TF-IDF features in GNB classifier generates improved results.

4.9 Performance summary and conclusion

4.10 Results Evaluation

In this section, we aim to evaluate our results based on evaluation metrics, mentioned in Chapter 3. and provide a complete assessment of the different experiments carried out in this research.

4.10.1 Comparison of independent and combined text features

It can be observed that using combined text features improves the accuracies of the machine learning models when compared to usage of independent text features. A list showing the average performance in 4 major evaluation metrics is shown in Table 4.12. In the table, it is seen that the highest score for ROC is 0.73, for accuracy it is 0.76, for micro avg F1 it is 0.6 and for weighted avg F1 it is 0.71. When these

values are compared to those obtained using a combination of features , Table 4.13, the highest values of ROC score, accuracy, micro average f1 score and weighted average f1 scores are 0.82, 0.81, 0.69 and 0.79 respectively. These results are 12.33%, 6.58%, 15% and 11.27% higher than those achieved by using independent features allowing us to concretely conclude that using a combination of text-based features improves classification results of machine learning models significantly.

Another observation made through the comparison of independent and combined text features was that out of all machine learning models used, MNB performs the best with the use of independent features whereas GNB works well with a combination of features. This helps improve our results further and hence it can for a baseline to be used in further research's in the future.

	ROC	**Accuracy**	**Micro avg F1**	**Weighted avg F1**
MNB	**0.73**	**0.76**	0.55	**0.71**
GNB	0.66	0.68	**0.6**	0.69
DT	0.47	0.58	0.49	0.63
KNN	0.56	0.71	0.51	0.68

Table 4.12: Average performance for all classifiers using independent features

	ROC	**Accuracy**	**Micro avg F1**	**Weighted avg F1**
MNB	0.77	0.76	0.43	0.65
GNB	**0.82**	**0.81**	**0.69**	**0.79**
DT	0.60	0.74	0.60	0.72
KNN	0.65	0.75	0.57	0.72

Table 4.13: Average performance for all classifiers using a combination of features

4.10.2 Comparison of text and graph modelling

An important purpose of this research is to determine which method is more accurate, text-based modelling or graph-based. Since many experiments were performed in the text-based part, it was decided to use combination of features in GNB classifier for comparison since it had the finest performance. The average of all values for each evaluation metrics was taken and a separate table was constructed. The comparison can be seen in Figure 4.2.

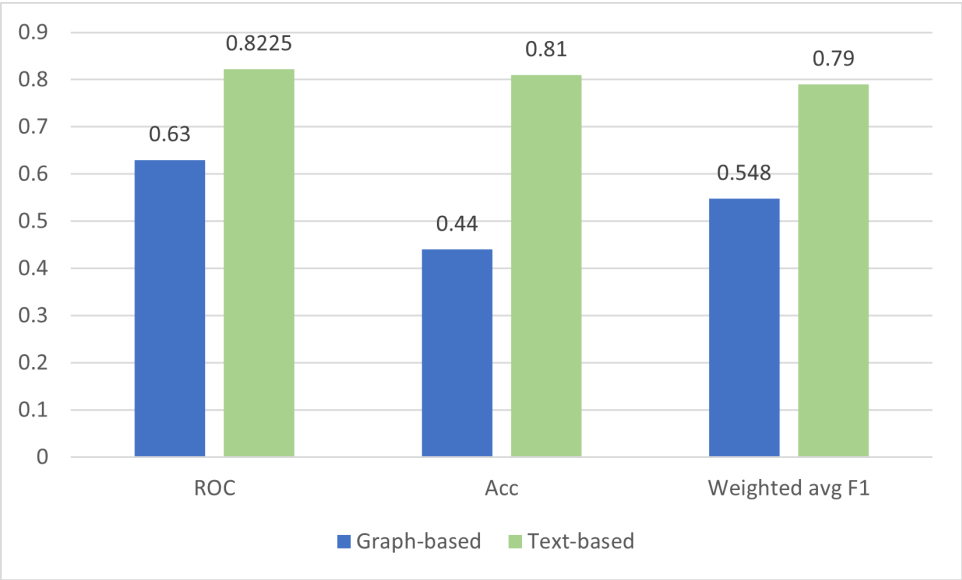

Figure 4.2: Comparison of graph-based and text-based modelling

In the figure above, it can be clearly seen that text-based modelling outperforms graph-based modelling in terms of ROC score, accuracy as well as weighted average F1 score. In the graph-based experiment,

an ROC score of 0.63 was obtained which is 30.56% higher than that of the text-based experiment. In terms of accuracy, a score of 0.44 was obtained by the graph-based experiment which is approximately 84% higher than that of text-based methods. Lastly, the weighted average also witnesses an increase of about 44% when the 2 are compared.

These results allow us to verify our initial research question for the value of a social network in detecting fake news. Based on the results of this research, text-based modelling produces more accurate results than graph-based when used with a supervised classifier. However, this may change based on the graph-based method being used.

4.10.3 Comparison with related work

Lastly, a comparison of this research was done with that of Abdullah Hamid [16], in which they analyse tweets related to COVID-19 and 5G conspiracy theories in an attempt to detect misinformation. They utilized BoW and BERT model-based approaches on their dataset, whereas numerous other methods were incorporated in this research such as TF-IDF and Word2Vec. Furthermore, the authors only applied Naive Bayes classifier on the BoW based model however a variety of other machine learning models were applied in this research such as SVM, KNN, DT, GNB and MNB. The wiser range of models being used allowed my results to be slight more reliable, experimentally. Other than this, graph representation has also been worked upon in this study in which they used Graph Neural Networks (GNNs) whereas in this research, non-graph classifiers with Node2Vec feature representation were used. This may be a disadavantage since results are optimzied using neural networks and more accurate outputs can be obtained. To evaluate their results, 2 metrics were used, F1 score and ROC score. For text based experiments, they achieved an average F1 score of 0.62 and an average ROC score of 0.36. In my case, ROC score, F1 score as well as accuracy were used which ,again, helped in validating my experiments further.

4.11 Assessment with regards to project objectives

The main aim of this project was to compare different ML classification models using two techniques to solve the issue of wrongful information being distributed during COVID-19.

During my research, I managed to achieve all objectives stated in Chapter 1 and mentioned below:

- The dataset is not labelled, adding binary labels to the dataset as true/fake is required
- Building a baseline model that classifying each tweet and improving its performance with coming experiments.
- Encoding BoW, TF-IDF and Word2Vec feature representation from the tweet text
- Training classifiers with independent feature representation
- Training more than one feature to represent the text
- Represent the tweet text related Information in a graph structure
- Get input vectors from the graph
- Training classifiers with graph feature representations
- Compare and analyse the performance of both text and graph approaches
- Applying graph feature representation to a GNN algorrithm

However, due to critical project time issues, the last objective involving GNN model remained incomplete. This can be done in the future, improving the practicality of this project.
[1]

[1] GitHub Repository Link https://github.com/AlmalkiHub/DSP

Chapter 5

Conclusion

The current pandemic has resulted in an acceleration of the usage of technology, on a government level as well as on a personal one. Information is being diffused at an uncontrollable speed and whereas majority of it is extremely useful as well as necessary, some of it is fabricated, resulting in life-threatening outcomes. In this research, I have explored the problem of COVID-19 misinformation during the pandemic by building an automatic COVID 19 misinformation detection system. A comprehensive comparison is presented between different ML classifiers using various input features for text based and graph structure based architectures.

In this research, a combination of 7 datasets, was initially cleaned, labelled and separated from JSON data files into CSV files to be used for each experiment. After the process of data annotation and pre-processing, the results in this research were generated for 3 scenarios and subsequently compared. The scenarios were text based classification models with independent features, text based classification models with combined features and graph structure classification.

During the evaluation of usage of independent features, it was observed that MNB performed the best for BoW and TF-IDF based features in terms of ROC, accuracy and F1-score. When Word2Vec based feature was used, GNB outperformed the other classifiers by obtaining a higher accuracy, F1 score and ROC score.

Secondly, when features were combined, SVM outperformed the other classifiers in all evaluation metrics when given combinations of BoW with TF-IDF and BoW with Word2Vec.

5.1 Summary

This thesis can be summarized in the following manner:

Chapter 1 provides an introduction to the non-technical aspects of this thesis. It explains the importance of this research as well as the motivation behind it whilst also discussing the various challenges linked with the project.

Chapter 2 contains a detailed explanation of all technical aspects which need to be understood for this research. A sufficient amount of literature review is provided for various machine learning techniques used for textual data and necessary terminologies linked with this research are explained. This chapter also includes details regarding the dataset to be used for our experiments.

Chapter 3 provides detailed descriptions of the experiments performed in this research. All steps performed to achieve the output are listed in a systematic manner and are explained conspicuously. This chapter also includes explanations of the machine learning algorithms to be used and lastly all evaluation metrics used are discussed in-depth.

Chapter 4 is an extremely crucial chapter since it contains all results obtained in this research and a thorough comparison of text-based data and graph-based data using 5 different machine learning algorithms is provided as well. The results are analyzed and any shortcomings are mentioned.

5.2 Project Status

The initial objectives of this project were to:

- Labelling the dataset, adding binary labels to the dataset as true/fake is required. (Complete)
- Building a baseline model that classify each tweet and improving its performance with coming experiments. (Complete)
- Encoding BoW, TF-IDF and Word2Vec feature representation from the tweet text. (Complete)
- Training classifiers with independent feature representation. (Complete except SVM classifier with independent features)
- Training more than one feature to represent the text. (Complete)
- Represent the tweet text related information in a graph structure. (Complete)
- Get input vectors from the graph. (Complete)
- Training classifiers with graph feature representation. (Complete)
- Compare and analyse the performance of both text and graph approaches. (Complete)
- Applying graph feature representation to a GNN algorithm. (Incomplete)

Some obstacles I faced during the execution of the research implementation include the following:

- Running the following experiments:
  - SVM algorithm with independent features including BoW, TF-IDF and Word2Vec : This could not be achieved because the model took more than 48 hours to train on local machine with great difficulty.
  - Combination of BoW, TF-IDF, Word2Vec based features: This also proved to be problematic as training took a long time.

5.3 Further Work

This research provides a foundation for machine learning based approaches to handle incorrect information during COVID-19. It can be extended in multiple directions, enhancing its practicality in real-world problems.

Firstly, the graph based aspect of this research can be extended by using different graph structure types and analyzing the results. It is highly possible that other types may improve the results further.

Another direction can be to alter the labelling process of the data. In this research, data has been annotated automatically using 4 hashtag keywords. In the future, this process can be performed manually to achieve more accurate results

Lastly, subsequent to machine learning algorithms, deep learning techniques have been increasingly used. These techniques have been providing much more accurate results and have the ability to self-learn, executing complex tasks efficiently. For this specific problem, deep neural networks can be utilized to improve the results substantially in a shorter amount of time. Specifically, Node2Vec embeddings can be performed using graph neural networks since they are highly capable of understanding complex patterns in data.

Bibliography

[1] Worldometers coronavirus cases. 2021.

[2] Wasim Ahmed, Josep Vidal-Alaball, Joseph Downing, Francesc López Seguí, et al. Covid-19 and the 5g conspiracy theory: social network analysis of twitter data. *Journal of medical internet research*, 22(5):e19458, 2020.

[3] Bilal Al-Ahmad, Ala'M Al-Zoubi, Ruba Abu Khurma, and Ibrahim Aljarah. An evolutionary fake news detection method for covid-19 pandemic information. *Symmetry*, 13(6):1091, 2021.

[4] Md Al-Zaman et al. Covid-19-related social media fake news in india. *Journalism and Media*, 2(1):100–114, 2021.

[5] Vimala Balakrishnan and Ethel Lloyd-Yemoh. Stemming and lemmatization: a comparison of retrieval performances. 2014.

[6] J Scott Brennen, Felix Simon, Philip N Howard, and Rasmus Kleis Nielsen. Types, sources, and claims of covid-19 misinformation. *Reuters Institute*, 7:3–1, 2020.

[7] Limeng Cui and Dongwon Lee. Coaid: Covid-19 healthcare misinformation dataset. *arXiv preprint arXiv:2006.00885*, 2020.

[8] Padraig Cunningham and Sarah Jane Delany. k-nearest neighbour classifiers: (with python examples). *arXiv preprint arXiv:2004.04523*, 2020.

[9] CIPRIAN-GABRIEL Cușmaliuc, LUCIA-GEORGIANA Coca, and ADRIAN Iftene. Identifying fake news on twitter using naive bayes, svm and random forest distributed algorithms. In *Proceedings of The 13th Edition of the International Conference on Linguistic Resources and Tools for Processing Romanian Language (ConsILR-2018) pp*, pages 177–188, 2018.

[10] Karen M Douglas. Covid-19 conspiracy theories. *Group Processes & Intergroup Relations*, 24(2):270–275, 2021.

[11] K SRIVASTAVA Durgesh and B Lekha. Data classification using support vector machine. *Journal of theoretical and applied information technology*, 12(1):1–7, 2010.

[12] Theodoros Evgeniou and Massimiliano Pontil. Support vector machines: Theory and applications. In *Advanced Course on Artificial Intelligence*, pages 249–257. Springer, 1999.

[13] Aditya Grover and Jure Leskovec. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 855–864, 2016.

[14] Stefano Guarino, Noemi Trino, Alessandro Chessa, and Gianni Riotta. Beyond fact-checking: Network analysis tools for monitoring disinformation in social media. In *International Conference on Complex Networks and Their Applications*, pages 436–447. Springer, 2019.

[15] Kriti Gupta. Fake news analysis and graph classification on a covid-19 twitter dataset. 2021.

[16] Abdullah Hamid, Nasrullah Shiekh, Naina Said, Kashif Ahmad, Asma Gul, Laiq Hassan, and Ala Al-Fuqaha. Fake news detection in social media using graph neural networks and nlp techniques: A covid-19 use-case. *arXiv preprint arXiv:2012.07517*, 2020.

[17] William L Hamilton. Graph representation learning. *Synthesis Lectures on Artifical Intelligence and Machine Learning*, 14(3):1–159, 2020.

[18] Yi Han, Shanika Karunasekera, and Christopher Leckie. Graph neural networks with continual learning for fake news detection from social media. *arXiv preprint arXiv:2007.03316*, 2020.

[19] Nitin Hardeniya, Jacob Perkins, Deepti Chopra, Nisheeth Joshi, and Iti Mathur. *Natural language processing: python and NLTK*. Packt Publishing Ltd, 2016.

[20] Rebecca Heilweil. Twitter now labels misleading coronavirus tweets with a misleading label, May 2020.

[21] Rajni Jindal, Ruchika Malhotra, and Abha Jain. Techniques for text classification: Literature review and current trends. *webology*, 12(2), 2015.

[22] Devyani Keskar, Sushila Palwe, and Ayushi Gupta. Fake news classification on twitter using flume, n-gram analysis, and decision tree machine learning technique. In *Proceeding of International Conference on Computational Science and Applications*, pages 139–147. Springer, 2020.

[23] Hyunuk Kim and Dylan Walker. Leveraging volunteer fact checking to identify misinformation about covid-19 in social media. *Harvard Kennedy School Misinformation Review*, 1(3), 2020.

[24] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.

[25] Jan Kirchner and Christian Reuter. Countering fake news: A comparison of possible solutions regarding user acceptance and effectiveness. *Proceedings of the ACM on Human-computer Interaction*, 4(CSCW2):1–27, 2020.

[26] Ramez Kouzy, Joseph Abi Jaoude, Afif Kraitem, Molly B El Alam, Basil Karam, Elio Adib, Jabra Zarka, Cindy Traboulsi, Elie W Akl, and Khalil Baddour. Coronavirus goes viral: quantifying the covid-19 misinformation epidemic on twitter. *Cureus*, 12(3), 2020.

[27] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

[28] Azzam Mourad, Ali Srour, Haidar Harmanai, Cathia Jenainati, and Mohamad Arafeh. Critical impact of social networks infodemic on defeating coronavirus covid-19 pandemic: Twitter-based study and research directions. *IEEE Transactions on Network and Service Management*, 17(4):2145–2155, 2020.

[29] Arjun Mukherjee, Vivek Venkataraman, Bing Liu, and Natalie Glance. What yelp fake review filter might be doing? In *Seventh international AAAI conference on weblogs and social media*, 2013.

[30] Harsh H Patel and Purvi Prajapati. Study and analysis of decision tree based classification algorithms. *International Journal of Computer Sciences and Engineering*, 6(10):74–78, 2018.

[31] Parth Patwa, Shivam Sharma, Srinivas Pykl, Vineeth Guptha, Gitanjali Kumari, Md Shad Akhtar, Asif Ekbal, Amitava Das, and Tanmoy Chakraborty. Fighting an infodemic: Covid-19 fake news dataset. In *International Workshop on Combating On line Ho st ile Posts in Regional Languages dur ing Emerge ncy Si tuation*, pages 21–29. Springer, 2021.

[32] Juan Pizarro. Using n-grams to detect fake news spreaders on twitter. In *CLEF (Working Notes)*, 2020.

[33] Adi Prasetyo, Bayu Dwi Septianto, Guruh Fajar Shidik, and Ahmad Zainul Fanani. Evaluation of feature extraction tf-idf in indonesian hoax news classification. In *2019 International Seminar on Application for Technology of Information and Communication (iSemantic)*, pages 1–6. IEEE, 2019.

[34] Lotte Pummerer, Robert Böhm, Lau Lilleholt, Kevin Winter, Ingo Zettler, and Kai Sassenberg. Conspiracy theories and their societal effects during the covid-19 pandemic. *Social Psychological and Personality Science*, page 19485506211000217, 2020.

[35] Shahzad Qaiser and Ramsha Ali. Text mining: use of tf-idf to examine the relevance of words to documents. *International Journal of Computer Applications*, 181(1):25–29, 2018.

[36] Irina Rish et al. An empirical study of the naive bayes classifier. In *IJCAI 2001 workshop on empirical methods in artificial intelligence*, volume 3, pages 41–46, 2001.

[37] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. The graph neural network model. *IEEE transactions on neural networks*, 20(1):61–80, 2008.

[38] Yunita Dwi Setiyaningrum, Annisa Fitrianingtiyas Herdajanti, Catur Supriyanto, et al. Classification of twitter contents using chi-square and k-nearest neighbour algorithm. In *2019 International Seminar on Application for Technology of Information and Communication (iSemantic)*, pages 1–4. IEEE, 2019.

[39] Gautam Kishore Shahi, Anne Dirkson, and Tim A Majchrzak. An exploratory study of covid-19 misinformation on twitter. *Online social networks and media*, 22:100104, 2021.

[40] Manoj Kumar Sharma, Nitin Anand, Akash Vishwakarma, Maya Sahu, Pranjali Chakraborty Thakur, Ishita Mondal, Priya Singh, Ajith Sj, et al. Mental health issues mediate social media use in rumors: implication for media based mental health literacy. *Asian journal of psychiatry*, 53:102132, 2020.

[41] Mandeep Singh, Mohammed Wasim Bhatt, Harpreet Singh Bedi, and Umang Mishra. Performance of bernoulli's naive bayes classifier in the detection of fake news. *Materials Today: Proceedings*, 2020.

[42] Yan-Yan Song and LU Ying. Decision tree methods: applications for classification and prediction. *Shanghai archives of psychiatry*, 27(2):130, 2015.

[43] Sander van Der Linden, Jon Roozenbeek, and Josh Compton. Inoculating against fake news about covid-19. *Frontiers in psychology*, 11:2928, 2020.

[44] Youze Wang, Shengsheng Qian, Jun Hu, Quan Fang, and Changsheng Xu. Fake news detection via knowledge-driven multimodal graph convolutional networks. In *Proceedings of the 2020 International Conference on Multimedia Retrieval*, pages 540–547, 2020.

[45] Xichen Zhang and Ali A Ghorbani. An overview of online fake news: Characterization, detection, and discussion. *Information Processing & Management*, 57(2):102025, 2020.

[46] Yin Zhang, Rong Jin, and Zhi-Hua Zhou. Understanding bag-of-words model: a statistical framework. *International Journal of Machine Learning and Cybernetics*, 1(1-4):43–52, 2010.

[47] Xinyi Zhou and Reza Zafarani. A survey of fake news: Fundamental theories, detection methods, and opportunities. *ACM Computing Surveys (CSUR)*, 53(5):1–40, 2020.